CSC3002：Exam 原题 × Lecture 知识点

期中复习专题 • 2021 / 2022 Spring Final 选题

本讲义按“知识点 → 完整原题 → 详细解析 → lecture 定位与相关知识”组织。先学会读类型、对象与调用关系，再把这套模型用于真实考试题。每个专题先建立概念；每题完整保留英文题干、代码和选项，然后给出六步解析。两道 Q32 的 24 个待填空格另逐格解释，首行已填基址不计空格。

范围依据：没有找到明确的本学期 Midterm Review 或正式截止 lecture。用户提供的 CHEN 版第 1-7 讲作为首要版本；同教师 2026 春第 8-9 讲补齐类、模板和迭代器；第 10-11 讲作为暂定边界补充。本讲义不把“约前 11 讲”的估计写成教师确认考纲。两卷都是 Final，绝不改称 Mid。第 11 专题的两道位运算题属于旧 M10 的边界查漏，相关系统课件在第 14 讲，单独标明。

核心问题：一个表达式的类型是什么，它读取或修改哪个对象，按哪些规则产生结果？路线是：类型和表达式 → 函数接口与共享 → 字符串与流 → 对象/地址/指针 → 连续数组与步长 → 生命周期 → 类和虚调用 → 模板及迭代接口。题目相似时仍先看年份、类型和执行顺序。

覆盖统计：2 套不同 Final，各选 24 道，共 48 道原题；其中 46 道属于上述暂定复习范围、2 道为位运算边界补充。两道 Q32 各 12 个待填空格，共 24 格，均逐格解析，不把每格重复计为一道原题。其余选择/程序题没有原卷编号小问。两年内容相同的题仍保留各自题号与原文，在同一专题内讲解。

本讲义的答案是根据原题独立推导的教学答案；没有取得官方答案表。2022 Q4 的绝对措辞、两卷 Q21 的 OOP 特征口径、2022 Q29 对切片的暗示、2021 Q23 的库版本和 Q17 的有符号整数前提，均在相应位置说明，不将争议悄悄抹平。

2021 原卷带他人手写批注。正文忠实重排印刷题面，保留原题笔误和不严谨表述，批注不作为题干或官方答案。完整原件及版本说明见仓库 exams 目录。旧卷考试规则仅属于原卷，不代替用户当前课件写明的闭卷要求。

目录

CSC3002：Exam 原题 × Lecture 知识点 1

目录 2

阅读定位与课件版本 3

01 从类型出发读表达式 4

02 函数、参数、引用、const 与重载 7

03 字符、C 字符串与 std::string 12

04 流对象、引用返回与连续输出 15

05 对象、地址、指针与间接访问 17

06 数组、地址步长与指针自增 27

07 进制与可寻址空间 41

08 生命周期、动态内存与 static 43

09 类、复制、继承与虚函数 48

10 模板、迭代器与基础容器接口 56

11 边界补充：位运算提取颜色分量 60

原题覆盖索引与未纳入范围 63

课件回查：本讲没有用原题覆盖的其他内容 66

必要的语言与库核对来源 67

阅读定位与课件版本

页码一律从相应 PDF 第一页起算。U01-U07 是用户原 PPT 按一张一页导出的 PDF，故 PDF 第 k 页对应原 PPT 第 k 张；不会借用不同长度的网页版本页码。S08-S11 是 LGUHub 2026 春同教师原 PDF，书中引用其 PDF 顺序页，不把课件内“6.1”等教材章节号当成 lecture 编号。没有统一连续印刷页码的页面以 PDF 顺序页为准。

U 代表用户提供版，S 代表同教师补充版；二者身份在下面逐个列明。每题解析后用短代号定位，完整文件名在此表及仓库 lecture 索引中可直接查到。

代号	文件名与主题	页数
U01	U01_1-CSC3002_Lecture Notes Overview of C++(5).pdf	64
U02	U02_2-CSC3002_Lecture Notes Functions and Libraries Part 1(2).pdf	41
U03	U03_3-CSC3002_Lecture Notes Functions and Libraries Part 2.pdf	34
U04	U04_4-CSC3002_Lecture Notes_Strings.pdf	39
U05	U05_5-CSC3002_Lecture Notes Streams(3).pdf	40
U06	U06_6-CSC3002_Pointers and Arrays(4).pdf	46
U07	U07_7-CSC3002 Dynamic memory management.pdf	30
S08	S08_8-CSC3002_Designing Classes.pdf	65
S09	S09_9-CSC3002_Template and Iterator(1).pdf	33
S10	S10_10_Vector & Stack(2).pdf	30
S11	S11_11-CSC3002_List&Queue.pdf	39

原卷 E21：CSC3002_Final_2021_Spring.pdf，封面 FINAL EXAMINATION / Term 2, 2020-2021，8 页、32 题、150 分钟。原卷 E22：CSC3002_Final_2022_Spring.pdf，封面 FINAL EXAMINATION / Term 2, Spring, 2021-2022，8 页、32 题、120 分钟。

原卷共用作答说明：Q1-Q10 为判断题，每题 1%，选 a=True、b=False；Q11-Q20 为单选，每题 2%；Q21-Q30 为多选，每题 4%，允许 1-4 个正确选项，原卷说明漏选可给部分分 2%；Q31 为格式化输出，12%；Q32 为 12 个待填表达式，各 1.5%，共 18%。各题保留自己的题号及选项；分值以这份说明和原题明确标注为准。

详细范围证据见 范围与来源。阅读建议：若卡在某题，先回到该专题开头的概念，按“类型 → 分组 → 对象关系 → 状态变化 → 结果格式”重新推一次。

01 从类型出发读表达式

知识点先讲

这一部分解决的问题是：同样一串数字，为什么把 `9` 改成 `9.0`，结果就会变？C++ 的计算规则由每一步操作数的类型决定。

变量是一个有类型、能保存值的对象；名字是访问它的方式。`int total = 42;` 中，`int` 是类型，`total` 是名字，`=` 在这里引入初始化，`42` 是初值，`;` 结束声明。类型决定能做哪些操作、如何解释存下来的内容。一个表达式也有类型和值，例如 `2 + 3` 的类型是 `int`，值是 5。

变量名用来让程序准确指认对象。先检查三个条件：是不是保留关键字；首字符是不是允许的字母或下划线；后面是否只含允许的字母、数字和下划线。大小写不同是不同名字。`type_name` 合法，`2name` 不能作为一个标识符，`type-name` 会被读成减法而不是一个变量名。初学示例优先用 ASCII 字母，实际 C++ 也允许部分 Unicode 标识符。

计算时先画括号结构，再在每个运算节点标类型。`*`、`/`、`%` 同优先级，按左结合理解；它们高于 `+`、`-`。这里的结合方向决定语法分组，不等于对任意带副作用程序承诺运行求值顺序。

```
int a = 9 / 6;          // 两个 int，相除结果为 int 1
double b = 9 / 6;       // 先得到 1，再转成 double 1.0
double c = 9.0 / 6;     // 此次除法按 double 计算，得到 1.5
```

`double b` 不能逆转右边已经发生的整数除法。要保留小数，必须在除法发生前让相应操作数成为浮点数，例如 `9.0` 或 `static_cast<double>(9)`。对本讲正整数，整数除法丢掉小数部分；一般规则是向零截断。`%` 取整数除法的余数，所以 `20 % 3` 是 2。

最小对照：`9*(8-7)/6` 得到 1，`9.0*(8-7)/6` 得到 1.5。下一步无论加什么浮点数，前一种算法丢掉的 0.5 都不会自动回来。

2021 Spring Final • Q24 • 标识符合法性

来源：E21，原卷 PDF 第 6 页；真实卷别 Final。

完整 exam 原题

24. Indicate which of the following are illegal variable names in C++:

a. typename

b. type_name

c. type-name

d. 2typenames

详细解析

题目在说什么

本题要从四个候选名字中选出不能作为 C++ 变量名的项，是多选题。检查的是词法规则，不是名字是否好听或是否能说明用途。

第一反应

先看关键字，再看首字符，最后看内部字符；对每个选项独立检查，不能找到一个错项就停止。

核心公式与规则

本题适用规则：保留关键字不能另作变量名；数字不能作首字符；连字符 - 是运算符，下划线 _ 可以出现在普通名字中。

推导

a 的 `typename` 是模板语法使用的关键字，不能另声明为变量。b 的 `type_name` 以字母开头，内部下划线合法。c 的 `type-name` 会拆成 `type`、减号、`name`，不是单个名字。d 的 `2typenames` 以数字开头，非法。因此 a、c、d 全部应选。

考场写法

选 a、c、d。`typename` is a keyword; `type-name` contains an operator; `2typenames` starts with a digit. `type_name` is a valid identifier.

易错点

把 `typename` 和 `type_name` 当成同一个词会漏选 a；把英文单词之间的连字符当成下划线会漏选 c；“后面有字母”不能弥补 d 的非法首字符。

Lecture 定位与这一块还包含什么

U1 第 26-28 页/张：Variables。还应掌握声明先于使用、类型与初值、大小写敏感、作用域；不要把命名风格建议当作语法禁止。

2021 Spring Final • Q11 • 整数除法先发生

来源：E21，原卷 PDF 第 3 页；真实卷别 Final。

完整 exam 原题

11. Calculate the result of the following expressions based on the C++ rules:

9 * (8-7)/6 + 5*4 % 3*(2+1.0)

a. 3

b. 3.5

c. 7

d. 7.5

详细解析

题目在说什么

题目要求按 C++ 规则算完整表达式 `9 * (8-7)/6 + 5*4 % 3*(2+1.0)`。左半边全为整数，右边最后一次乘法会遇到浮点数。

第一反应

不要看到尾部 `1.0` 就把整条表达式提前当成浮点计算。先把最外层加号左右两边分别计算。

核心公式与规则

整数与整数相除产生整数结果；同优先级的 `* / %` 左结合；只在某次算术运算有浮点操作数时，才对该次运算进行相应转换。

推导

括号先得到 `(8-7)=1`，`(2+1.0)=3.0`。左边按 `(9*1)/6=9/6=1`。右边按 `((5*4)%3)*3.0=(20%3)*3.0=2*3.0=6.0`。最外层为 `1+6.0=7.0`，与选项 c 的 7 对应。

考场写法

选 c。左项为整数 1；右项为 double 6.0；总结果为 double 7.0。

易错点

误答 7.5 是把左边 `9/6` 按实数除法算了；误以为赋给 double 能修复也不对。`%3*(...)` 要先取余，再乘括号，不能把整个后半串当作模数。

Lecture 定位与这一块还包含什么

U1 第 35-44 页/张：Expressions、Division and Type Casts、Remainder、Precedence。还应掌握显式转换、整数除法、括号、左结合以及类型转换发生的位置。

2022 Spring Final • Q11 • 浮点字面量改变局部运算

来源：E22，原卷 PDF 第 3 页；真实卷别 Final。

完整 exam 原题

11. Calculate the result of the following expressions based on the C++ rules:

9.0*(8-7) / 6 + 5*4 % 3*(2+1)

a. 3

b. 3.5

c. 7

d. 7.5

详细解析

题目在说什么

本卷是 `9.0*(8-7) / 6 + 5*4 % 3*(2+1)`。它与 2021 卷只需比较两个局部：左端是 `9.0`，右侧括号是整数 `2+1`。

第一反应

从叶子操作数向上标类型。`9.0` 使左边乘法和随后的除法按浮点运算，右边取余部分仍是整数。

核心公式与规则

浮点数与整数作通常算术运算时，本例整数转换为 double；余数运算 `%` 的操作数必须是整数类型。

推导

`8-7=1`，所以左边 `9.0*1/6=9.0/6=1.5`。`5*4=20`，`20%3=2`，`2+1=3`，右边为 `2*3=6`。最后 `1.5+6=7.5`。

考场写法

选 d，结果 7.5。关键区别在 `9.0`：本次除法不发生整数截断。

易错点

沿用 2021 卷答案 7 会漏看字面量类型；先算 `3*(2+1)` 再取余会破坏原表达式分组。用括号把每一步写清，比只背优先级口诀可靠。

Lecture 定位与这一块还包含什么

U1 第 35-44 页/张，主题同前题；第 38-40 页的整数除法对照尤其直接。还应练习只改变一个字面量的最小变式。

02 函数、参数、引用、const 与重载

知识点先讲

函数把一种操作封装成可调用的单位，解决重复代码与复杂步骤难以组织的问题。先看一段最小定义：

```
int addOne(int x) {
    return x + 1;
}
int n = 4;
int answer = addOne(n);
```

前面的 `int` 是返回类型，`addOne` 是函数名，圆括号中的 `int x` 是形参声明，花括号包住函数体，`return` 把结果交回调用位置。调用中的 `n` 是实参表达式。这里先读出 n 的值 4，再用它初始化独立的形参 x；返回 5 后，answer 初始化为 5，n 仍是 4。parameter 是被调用函数声明的形参，argument 是调用时提供的实参。

函数可以没有参数：`int getOne() { return 1; }`；也可以不返回值：`void hello() { std::cout << "Hi"; }`。`void` 函数仍然会执行并返回调用处，只是不给调用者一个结果值。函数可以写多个 `return` 作为不同路径的出口；一次调用执行到其中一个出口后就结束，不会继续执行后面的出口。

传值适合得到独立副本。若要让函数修改外面的对象，要建立共享关系：

```
void byValue(int x) { x = 9; }
void byReference(int& x) { x = 9; }
int n = 4;
byValue(n);       // 改局部副本，n 仍是 4
byReference(n);   // x 绑定 n，n 变成 9
```

声明中的 `&` 表示引用类型。引用是访问同一对象的另一个名字；绑定后执行 `x=9` 是写入已绑定的对象，不是改绑。大对象只读时，`const T&` 同时表达“共享对象”与“不通过这个形参修改它”：

```
std::size_t lengthOf(const std::string& text) {
    return text.size();
}
```

`const` 在这里约束 text 这条访问路径，并不把原来可修改的调用者对象永久冻结。`const T*` 也可表达只读指针，但题目若问课件里的 constant call by reference，应对应 `const T&`。

声明告诉编译器函数如何被调用，定义提供函数体；signature 则用于区分同名函数。本讲讨论同一作用域中的普通非模板函数：参数类型和顺序、参数个数影响重载；参数名字不影响；只改返回类型不能构成重载。

```
int f(int x);
int f(double x);       // 合法的另一个重载
int f(int renamed);    // 同一个函数的再次声明
// double f(int x);    // 与第一项仅返回类型不同，非法
```

按值参数的顶层 const，例如 `f(int)` 与 `f(const int)`，也不能形成两个重载。返回类型仍然重要，只是它不能单独帮这个调用选择重载。不要把这条初级规则无条件推广到模板和全部成员函数签名细节。

2021 Spring Final • Q2 • 多个 return 与一次调用的出口

来源：E21，原卷 PDF 第 2 页；真实卷别 Final。

完整 exam 原题

2. In C++, a function is a named section of code that performs a specific operation, and a function can include only one return statement as the exit point of the function.

a. True b. False

详细解析

题目在说什么

判断句说一个函数的代码里只能出现一个 return。它把“源代码中能写几个出口”和“一次调用实际退出几次”混在一起。

第一反应

看到 only one，先找合法反例。按条件分别返回正负或真假，就是多个 return 的常见用途。

核心公式与规则

`return` 结束当前函数调用；控制流可以在到达出口前分支。多个互斥路径各自包含 return 完全合法。

推导

例如 `int sign(int x) { if (x>0) return 1; if (x<0) return -1; return 0; }` 有三个 return。输入 3 时第一个 return 执行，函数已经结束；输入 -2 时走到第二个；输入 0 时走到第三个。每次只取一条路径，不妨碍代码中有三个出口。

考场写法

选 b，False。A function may contain multiple return statements; executing one terminates the current call.

易错点

不要把“本次调用只退出一次”误写成“源代码只能有一个 return”。也不要认为 return 只退出最近的 if 或循环；它退出的是整个当前函数调用。

Lecture 定位与这一块还包含什么

U2 第 8-9、24 页/张：函数定义、返回与调用过程；U3 第 18 页/张的 Direction 转字符串函数也有多个 return。还应区分 return、break、void 与返回值。

2022 Spring Final • Q2 • 函数不必同时有输入参数和返回值

来源：E22，原卷 PDF 第 2 页；真实卷别 Final。

完整 exam 原题

2. In C++, a function is a named section of code that performs a specific operation, and a function must have a parameter list as the input and return a value as the output.

a. True b. False

详细解析

题目在说什么

题干前半句给函数一个直观描述，后半句宣称函数必须有参数作为输入且必须返回值。整个判断句只要有一部分错误，就应选 False。

第一反应

检查两个 must。分别构造无参数函数和 void 函数，即可判断这个全称断言。

核心公式与规则

空参数列表 `()` 在 C++ 定义中可以表示不接收参数；返回类型 `void` 表示不返回值。参数传入和返回值传出是函数接口的两种方式，但不是每个函数都必须具备。

推导

`int getOne() { return 1; }` 没有输入参数，但合法。`void setZero(int& x) { x=0; }` 通过引用修改对象，没有返回值，也合法。`void hello() { std::cout << "Hi"; }` 两者都没有，仍可完成输出操作。因此题干后半句不成立。

考场写法

选 b，False。Functions may have no parameters, and a void function returns no value.

易错点

“会返回调用者”不等于“会返回一个值”；void 也不等于函数什么都不做。不要被前半句基本正确的定义带着把整句话判真。

Lecture 定位与这一块还包含什么

U2 第 4-9、24 页/张：函数概念、参数、void 与调用机制。还应掌握声明/定义、实参/形参、作用域和副作用。

2021 Spring Final • Q20 • 大对象的只读共享

来源：E21，原卷 PDF 第 5 页；真实卷别 Final。

完整 exam 原题

20. If the parameter you are passing is an object whose value should not change, and you don’ t want to make a copy of the object because it occupies a lot of memory , which one of the following approaches should you use to pass this object?

a. Call by value

b. Call by reference

c. Call by pointer

d. Constant call by reference

详细解析

题目在说什么

实参是占用较多内存的对象。题目同时要求不能通过函数改变它的值，而且希望避免复制。要选同时满足两条要求的传参方式。

第一反应

先把需求拆开：“不复制”指向引用；“只读”给引用加 const。再对照选项 d 的 Constant call by reference。

核心公式与规则

参数 `const T& obj` 把引用绑定到实参，函数通过该引用只能进行 const 允许的操作。按值 `T obj` 则初始化一个独立对象，通常涉及复制或其他对象构造。

推导

a 传值不能表达本题避免副本的要求。b 普通 `T&` 虽不复制，却允许修改。c 仅说 pointer 并没有指定指向 const 对象，不能直接满足题目要求。d 明确对应 `const T&`，既避免作为参数创建大对象副本，又提供只读接口。

考场写法

选 d：constant call by reference，例如 `void inspect(const BigObject& obj);`。

易错点

不要只盯效率而选 b，也不要把 `const T&` 解释成“复制一份常量”。如果自己设计接口，`const T*` 也能做到只读间接访问；但这不等于原题笼统的 call by pointer 就是最明确选项。

Lecture 定位与这一块还包含什么

U2 第 27-31 页/张：引用与消除复制；U7 第 22 页/张：Constant call by reference。还应掌握 const 对访问路径的约束、可修改引用、引用绑定与对象生命周期。

2021 Spring Final • Q22 • signature 到底比较什么

来源：E21，原卷 PDF 第 5 页；真实卷别 Final。

完整 exam 原题

22. In C++, functions can be overloaded, which means that you can define different functions with the same name as long as they have different signatures. The signature of a function includes the following:

a. The number of the parameters

b. The names of the parameters

c. The types of the parameters

d. The type of the return value

详细解析

题目在说什么

本题是多选：同名普通函数允许重载，要求判断用于区分它们的 signature 包含哪些信息。四项分别是参数数量、参数名字、参数类型、返回类型。

第一反应

从调用端想：编译器看到函数名和实参列表，可以据此匹配形参类型，却看不到你在定义里给形参取的名字。再检查返回类型能不能单独区分同一次调用。

核心公式与规则

在本题的普通非模板函数语境中，同名函数的参数类型序列用于区分重载。序列长度就是参数数量；参数名字不构成区别；只有返回类型不同不能新增重载。

推导

a：`f(int)` 与 `f(int,int)` 数量不同，能区分。b：`f(int x)` 与 `f(int y)` 同一声明，名字不区分。c：`f(int)` 与 `f(double)` 参数类型不同，能区分。d：`int f(int)` 与 `double f(int)` 无法只靠返回类型区分，是冲突声明。因此选 a、c。

考场写法

选 a、c。The parameter count and parameter types distinguish these overloads; parameter names and return type do not.

易错点

把形参改名不等于重载；给调用结果换个接收变量类型，也不会让“仅返回类型不同”的声明变合法。按值形参的顶层 const 也不是可用来区分的变化。

Lecture 定位与这一块还包含什么

U2 第 17-18 页/张：C++ Enhancements to Functions 与 signature 练习。还应掌握参数顺序、默认实参可能导致的调用歧义，以及声明合法和某次调用能唯一匹配的区别。

2022 Spring Final • Q22 • signature 到底比较什么

来源：E22，原卷 PDF 第 5 页；真实卷别 Final。

完整 exam 原题

22. In C++, functions can be overloaded, which means that you can define different functions with the same name as long as they have different signatures. The signature of a function includes the following:

a. The number of the parameters

b. The names of the parameters

c. The types of the parameters

d. The type of the return value

详细解析

题目在说什么

本题是多选：同名普通函数允许重载，要求判断用于区分它们的 signature 包含哪些信息。四项分别是参数数量、参数名字、参数类型、返回类型。

第一反应

从调用端想：编译器看到函数名和实参列表，可以据此匹配形参类型，却看不到你在定义里给形参取的名字。再检查返回类型能不能单独区分同一次调用。

核心公式与规则

在本题的普通非模板函数语境中，同名函数的参数类型序列用于区分重载。序列长度就是参数数量；参数名字不构成区别；只有返回类型不同不能新增重载。

推导

a：`f(int)` 与 `f(int,int)` 数量不同，能区分。b：`f(int x)` 与 `f(int y)` 同一声明，名字不区分。c：`f(int)` 与 `f(double)` 参数类型不同，能区分。d：`int f(int)` 与 `double f(int)` 无法只靠返回类型区分，是冲突声明。因此选 a、c。

考场写法

选 a、c。The parameter count and parameter types distinguish these overloads; parameter names and return type do not.

易错点

把形参改名不等于重载；给调用结果换个接收变量类型，也不会让“仅返回类型不同”的声明变合法。按值形参的顶层 const 也不是可用来区分的变化。

Lecture 定位与这一块还包含什么

U2 第 17-18 页/张：C++ Enhancements to Functions 与 signature 练习。还应掌握参数顺序、默认实参可能导致的调用歧义，以及声明合法和某次调用能唯一匹配的区别。

03 字符、C 字符串与 std::string

知识点先讲

字符是单个字符值，字符串是字符序列。`'a'` 是 char 字符字面量；`"a"` 是含字符 a 和终止符的字符数组字面量。`'\0'` 是值为零的字符，C 字符串用它表示结束；它不是字符 `'0'`。

```
char letter = 'a';
char text[] = "abc";          // a b c \0, 共四个 char
std::string name = "abc";     // 一个管理字符序列的 string 对象
```

声明中的 `[]` 表示数组。text 的字符槽位连续存储，数组长度包含终止符；`std::string::size()` 给出的逻辑长度则是 3。string 会管理自己的字符存储，并提供拼接、查找、截取等操作。类型不同，能用的操作也不同。

`+` 对 std::string 有拼接重载，但不是对所有看起来像字符串的东西都可用。单看一对操作数：至少一方是适当的 std::string 对象时，可以匹配相关拼接重载；两个普通字符串字面量参与 `+`，通常转换为两个指针，C++ 没有“指针加指针”的拼接规则。

```
std::string a = std::string("hello") + "," + "world";
std::string b = "hello" + std::string(",") + "world";
// std::string c = "hello" + "," + std::string("world");
```

前两行的第一次加法已有一个 string 对象，能形成合法结果再参加下一次加法。第三行按左结合解析，最先需要成立的是 `"hello" + ","`，它已经不合法；后面再出现 string 救不了前面。左边接收变量的类型也不能把非法右侧整体自动修好。

相邻字面量 `"hello" "," "world"` 是编译期字面量拼接，它没有加号，机制与上面的运算符重载不同。做输出题之前，先检查表达式是否合法；非法程序没有题目声称的运行结果。

2021 Spring Final • Q3 • 最后才转换成 string 来不及

来源：E21，原卷 PDF 第 2 页；真实卷别 Final。

完整 exam 原题

3. The following two statements will generate two strings with the same content:

```
std::string str1 = "hello,world";
std::string str2 = "hello" + "," + std::string("world");
```

a. True b. False

详细解析

题目在说什么

题干声称两条声明会生成内容相同的字符串。第一条直接用字面量初始化 str1；第二条把两个字面量相加后，再加 `std::string("world")`。

第一反应

先检查第二条能否编译，再考虑内容是否相同。把连续加号按左结合加上括号，会立即暴露出最早出错的位置。

核心公式与规则

`a+b+c` 按 `(a+b)+c` 分组。C++ 的 std::string 拼接重载不能让两个普通字符指针相加；两个字符串字面量也不会因为目标变量是 string 就提前变为 string。

推导

str1 的初始化合法，内容是 hello,world。str2 先要求计算 `"hello" + ","`；这两项都是字符数组字面量，会在相应表达式中退化为指针，没有合法的拼接加法。故声明编译失败，还没有一个可以与 str1 比较的 str2。若把第一个或第二个字面量显式转成 std::string，则表达式才能逐步成立。

考场写法

选 b，False。The second declaration is ill-formed because the first addition is between two string literals, not a string and a literal.

易错点

不要写成“运行后得到另一个字符串”，错误发生在编译阶段；不要认为最右边的 std::string 能向左追溯改变已分组的加法；也不要把相邻字面量的自动拼接与 `+` 混为一谈。

Lecture 定位与这一块还包含什么

U4 第 8-15 页/张：char 与 C 字符串；第 22-26 页/张：string 运算与字面量拼接。还应掌握终止符、长度/容量、下标、substr、find/npos，以及 getline 和按词输入的区别。

2022 Spring Final • Q3 • 最后才转换成 string 来不及

来源：E22，原卷 PDF 第 2 页；真实卷别 Final。

完整 exam 原题

3. The following two statements will generate two strings with the same content:

```
std::string str1 = "hello,world";
std::string str2 = "hello" + "," + std::string("world");
```

a. True b. False

详细解析

题目在说什么

题干声称两条声明会生成内容相同的字符串。第一条直接用字面量初始化 str1；第二条把两个字面量相加后，再加 `std::string("world")`。

第一反应

先检查第二条能否编译，再考虑内容是否相同。把连续加号按左结合加上括号，会立即暴露出最早出错的位置。

核心公式与规则

`a+b+c` 按 `(a+b)+c` 分组。C++ 的 std::string 拼接重载不能让两个普通字符指针相加；两个字符串字面量也不会因为目标变量是 string 就提前变为 string。

推导

str1 的初始化合法，内容是 hello,world。str2 先要求计算 `"hello" + ","`；这两项都是字符数组字面量，会在相应表达式中退化为指针，没有合法的拼接加法。故声明编译失败，还没有一个可以与 str1 比较的 str2。若把第一个或第二个字面量显式转成 std::string，则表达式才能逐步成立。

考场写法

选 b，False。The second declaration is ill-formed because the first addition is between two string literals, not a string and a literal.

易错点

不要写成“运行后得到另一个字符串”，错误发生在编译阶段；不要认为最右边的 std::string 能向左追溯改变已分组的加法；也不要把相邻字面量的自动拼接与 `+` 混为一谈。

Lecture 定位与这一块还包含什么

U4 第 8-15 页/张：char 与 C 字符串；第 22-26 页/张：string 运算与字面量拼接。还应掌握终止符、长度/容量、下标、substr、find/npos，以及 getline 和按词输入的区别。

04 流对象、引用返回与连续输出

知识点先讲

输出流是管理输出状态和目的地的对象。`std::cout << x;` 使用插入运算，把 x 的表示写到流里。`<<` 在整数位运算中可以表示移位，在流表达式中则参与重载；含义必须结合操作数类型判断。

让用户自定义的 Point 也能输出时，可写以下教学示例：

```
struct Point { int x, y; };
std::ostream& operator<<(std::ostream& out, const Point& p) {
    out << '(' << p.x << ',' << p.y << ')';
    return out;
}
```

`std::ostream&` 是返回类型，表示返回已有输出流的引用；`operator<<` 是重载函数名；`out` 以引用接收流，避免复制；`const Point& p` 只读共享待打印的点；`return out` 把同一条流继续交给后续输出。

`cout << p << '\n'` 按 `(cout << p) << '\n'` 分组。第一次操作返回 cout 的引用，所以第二次可以继续写同一条流。引用返回不仅关系效率，也关系对象身份、流状态和连续调用接口。

标准流不可复制；写成 `std::ostream copy = std::cout;` 不会成功制造一个副本。另一方面，现代 C++ 的某些具体流类型支持移动，可以按值返回一个新的或转移所有权的流对象。必须区分“把现有 ostream 作为常规输出操作的参数/返回值”与“构造并返回新的文件流等对象”。后者使旧讲义中所有流都绝对必须引用返回的表述过强。

2021 Spring Final • Q4 • 不能把不可复制的流说成会按值复制

来源：E21，原卷 PDF 第 2 页；真实卷别 Final。

完整 exam 原题

4. The insertion operator << is often overloaded in different classes to insert the contents of the classes into an output stream for display purposes. The overloading usually requires the use of return by reference, because it avoids making a copy of the output stream. If you use return by value, a new copy of the output stream will be returned.

a. True b. False

详细解析

题目在说什么

本题讨论重载 <<，声称引用返回避免复制，并进一步断言若按值返回就会产生一个输出流新副本。真正要核查的是最后这句话能否在标准流类型上成立。

第一反应

先识别流是不可复制对象。“按值通常可能复制”是普通对象经验，不能绕过 ostream 自己的复制限制。

核心公式与规则

典型输出运算符为 `std::ostream& operator<<(std::ostream&, const T&)`。返回类型上的 & 让结果仍指向同一流；std::ostream 的复制构造不可用。

推导

典型重载接收 cout，往其缓冲区写入内容，再返回 out 引用。若把这种重载返回类型改为 std::ostream 并保留 `return out;`，需要从现有流左值构造结果，不能使用被禁用的复制操作，因此并不会成功返回所谓“新副本”。题干把非法操作说成必然复制，整体为假。

考场写法

选 b，False。Standard output streams are non-copyable; returning the existing stream by value in this way is ill-formed, rather than producing a copy.

易错点

不要只读前半句“引用避免复制”就判真；不可复制不是“复制慢”。也不要把引用返回读成返回局部对象地址：此例返回的是调用者传进来的流，生命周期仍由调用者管理。

Lecture 定位与这一块还包含什么

U3 第 16、18 页/张：Direction 的 operator<<；U5 第 11、17、24 页/张：连续输出、Point 重载与流引用。还应掌握输入流 >>、文件流、getline、读入失败状态与 EOF。

2022 Spring Final • Q4 • 旧卷常规接口说法与现代 C++ 的边界

来源：E22，原卷 PDF 第 2 页；真实卷别 Final。

完整 exam 原题

4. In C++, streams are complicated objects that cannot be copied. If you want to pass a stream object into a function you must use call by reference, and if you want to return a stream object from a function, you must use return by reference.

a. True b. False

详细解析

题目在说什么

本卷判断句把“streams cannot be copied”与两个必须使用引用的说法连在一起。前者正确，后者需区分课程常规流接口与现代 C++ 的所有可能写法。

第一反应

先按课件使用场景判断：函数操作传入的既有流并返回同一流，应该用引用。再检查 must 是否被说成适用于任何流类型、任何返回情形。

核心公式与规则

常规接口：`void read(std::istream& in)` 与 `std::ostream& operator<<(std::ostream& out, const T&)`。但 C++11 起，一些具体流类型如 std::ifstream 支持移动；“不可复制”不推出“不可按值返回新流对象”。

推导

按课程所讲的已有流对象传入/传出模型，本题预期是 True：引用保持同一流的身份，避免非法复制。若逐字作为现代 C++ 的无条件命题，它过强，例如 `std::ifstream openFile() { return std::ifstream("data.txt"); }` 可以返回新文件流；也可把流的地址交给函数。因此需要同时知道课程意图和准确边界，不能把 must 背成语言定理。

考场写法

按本卷课件语境选 a（True），可写：Existing stream objects are conventionally passed and returned by reference because they cannot be copied. 严格讨论现代 C++ 时，应补充“某些具体流可移动、可按值返回新对象”，原断言不是无条件成立。

易错点

误区一：把 2021 Q4 的 False 直接搬来，忽视它明确说“会复制”，而本题问的是常规接口模型。误区二：把课程预期当成官方答案；本讲没有官方答案表。误区三：把 moving 与 copying 当成同一件事。

Lecture 定位与这一块还包含什么

U3 第 16、18 页/张；U5 第 24、30 页/张讲输入流按引用传递。补充边界以 C++ 标准对 basic_ostream 的复制限制、basic_ifstream 的移动构造为准；应掌握引用返回与生命周期、文件打开/关闭以及流状态。

05 对象、地址、指针与间接访问

知识点先讲

这一章的主问题是：表达式说的究竟是对象、对象里的内容，还是通往对象的地址？先建立两个对象，再画关系。

```
int x = 5;
int* p = &x;
```

x 是保存整数的对象。p 也是对象，但它的类型是 `int*`，保存指向 int 对象的指针值。表达式中的 `&x` 取得 x 的地址，用来初始化 p；声明中的 `*` 则是类型声明的一部分，不是此刻执行了一次解引用。

表达式	指认什么	示例状态
`x`	原来的 int 对象；取值时读其内容	5
`&x`	x 的地址	假设 0x1000
`p`	指针对象；取值时读其中的地址	0x1000
`*p`	顺着 p 访问到的 int 对象，即 x	取值为 5
`&p`	指针对象 p 自己的地址	假设 0x2000

地址数字只是教学假设，不保证真实机器如此分配。把 `*p=8` 读作“找到 p 指向的对象并写 8”，因此 x 变成 8，p 存的地址没有变。`p=&y` 则是在修改指针对象，让它以后指向 y。两句改的是不同对象。

声明逗号分隔的多个名字时，每个声明符都要独立读。`int* p, q;` 的共同基础类型是 int；p 的声明符带星号，q 没有，所以只有 p 是指针。空格不会代替第二颗星号。

再加一层：

```
int** pp = &p;
```

pp 保存 p 的地址，`*pp` 访问 p，`**pp` 再沿 p 访问 x。每解引用一次就沿箭头走一层。不要根据地址末尾数字猜测对象值；先读类型，再画 `pp -> p -> x`。

左值（lvalue）首先是一种表达式类别：它能指认一个对象或函数。对本章的普通对象例子，p 指认指针对象，`*p` 指认被指对象，因此它们是左值；`p+1` 计算出新的指针值，`&p` 产生地址值，本例都不是左值。注意“是左值”不保证“能赋值”：`const int k=3` 中的 k 是不可修改的左值，数组也有相应限制。

`*&x` 是先取 x 的地址再访问 x；对本章有效普通对象以及内建运算符，得到原对象。`&*p` 是先以 p 所指对象作为取地址目标，得到相应指针。课程判断题默认有效的普通对象指针；这个对照不能作为访问空指针、悬空指针或已结束生命周期对象的许可。

函数也可以由指针指认：`double (*f)(double)` 中先看 `(*f)`，说明 f 是指针；外侧 `(double)` 指向一种接收 double 参数的函数；最左 `double` 是该函数返回类型。若删掉星号外的括号写 `double *f(double)`，就变成“一个返回 double 指针的函数”。括号决定声明的结构。

2021 Spring Final • Q1 • 变量必须先声明

来源：E21，原卷 PDF 第 2 页；真实卷别 Final。

完整 exam 原题

1. In C++, a variable is a named address for storing a type of value, and you must declare a variable before you can use it.

a. True b. False

详细解析

题目在说什么

题目把变量描述为保存某种类型值的具名地址，并要求变量使用前先声明。它采用课件的“名字对应存储位置”教学模型。

第一反应

把题干对照 `int x=5;`：声明同时告诉编译器名字 x 和类型 int，后续语句才能按这个类型解释对 x 的使用。

核心公式与规则

声明建立名字及其类型；在某处使用名字时，合适的声明必须已经可见。对象、名字、地址和存储的值是彼此相关但不同的概念。

推导

在同一块中先写 `int x=5;` 再写 `cout<<x;` 可以找到声明。若第一次使用 x 时没有任何可见声明，编译器不能因为下一行才出现声明就反向补全。题目的两层意思与课件基础模型一致。更精确地说，变量是声明引入的对象或引用，地址是对象的位置，并不等于所有语言层面的变量定义。

考场写法

按课件模型选 a，True。A visible declaration establishes the variable name and type before use.

易错点

别把“声明先于使用”误成“必须在文件第一行定义全部变量”；局部块内也能声明。也别把名字、值和地址写成同一个东西，后续指针题会因此混乱。

Lecture 定位与这一块还包含什么

U1 第 26-30 页/张：Variables、Scope and Extent；U6 第 17-19 页/张：变量、地址与左值。还应掌握初始化/赋值、局部作用域以及同名变量的遮蔽。

2022 Spring Final • Q1 • 指针也是一个对象

来源：E22，原卷 PDF 第 2 页；真实卷别 Final。

完整 exam 原题

1. In C++, a variable is a named address for storing a type of value, and a pointer is a special type of variable that holds the address of another variable.

a. True b. False

详细解析

题目在说什么

本卷后半句讨论指针：它是一类保存另一对象地址的变量。题目要判断这种变量与指针的基础描述是否符合课程模型。

第一反应

画两个盒子。一个盒子 x 装整数，一个盒子 p 装能指向 x 的地址；p 自己也有存储位置。

核心公式与规则

`int* p=&x`：p 的类型是 int 指针，初始化值是 x 的地址。`p` 的值与 `&p` 的值属于不同层次。

推导

设 x 的值为 5、地址为 0x1000，p 自身位于 0x2000。则 p 存 0x1000，`*p` 访问 x 得 5，`&p` 为 0x2000。这实现了题干说的保存地址。指针还可能为空、指向函数等；题干是在介绍对象指针的典型用途，不能据此推论每个指针都始终指向有效变量。

考场写法

按本题基础模型选 a，True。A pointer variable stores a pointer value, typically the address of another object.

易错点

不要把 p 的值写成 x 的内容 5；不要把 p 与 &p 混同。指针声明本身不保证已初始化，也不保证其目标仍然有效。

Lecture 定位与这一块还包含什么

U6 第 17、20-24 页/张：变量、Pointers、Pointer Operators 与关系图。还应掌握 nullptr、未初始化指针、对象地址与指针自身地址。

2021 Spring Final • Q6 • 星号必须跟随每个声明符

来源：E21，原卷 PDF 第 2 页；真实卷别 Final。

完整 exam 原题

6. The following statements declare two pointers p1 and p3, and two integers p2 and p4:

```
int*  p1, p2;
int  *p3, p4;
```

a. True b. False

详细解析

题目在说什么

两行分别声明 p1/p2 和 p3/p4。要判定哪些名字具有指针类型，不能按星号离 int 近还是离名字近来猜。

第一反应

把每行在逗号处拆开，给每个名字重新补上基础类型 int。

核心公式与规则

`int* p1, p2;` 等价于分别声明 `int* p1; int p2;`；空格不改变语法。要两个都为指针，必须写 `int *p1, *p2;`。

推导

第一行 p1 的声明符含 *，故类型 int*；p2 的声明符不含 *，故类型 int。第二行 p3 的声明符含 *，故类型 int*；p4 不含 *，故类型 int。因此题干对四个变量的描述正确。

考场写法

选 a，True。p1、p3 是 int*；p2、p4 是 int。

易错点

把 `int*` 整块机械分发给逗号后所有名字，会错判 p2。空格位置只是排版。初学写代码可一行声明一个变量，阅读题目仍要会拆声明符。

Lecture 定位与这一块还包含什么

U6 第 22 页/张：Declaring a Pointer Variable。还应掌握指针初始化、指针的目标类型、多级指针；注意声明中的 * 与表达式中的 * 含义不同。

2022 Spring Final • Q6 • 星号必须跟随每个声明符

来源：E22，原卷 PDF 第 2 页；真实卷别 Final。

完整 exam 原题

6. The following statements declare two pointers p1 and p3, and two integers p2 and p4:

```
int*  p1, p2;
int  *p3, p4;
```

a. True b. False

详细解析

题目在说什么

两行分别声明 p1/p2 和 p3/p4。要判定哪些名字具有指针类型，不能按星号离 int 近还是离名字近来猜。

第一反应

把每行在逗号处拆开，给每个名字重新补上基础类型 int。

核心公式与规则

`int* p1, p2;` 等价于分别声明 `int* p1; int p2;`；空格不改变语法。要两个都为指针，必须写 `int *p1, *p2;`。

推导

第一行 p1 的声明符含 *，故类型 int*；p2 的声明符不含 *，故类型 int。第二行 p3 的声明符含 *，故类型 int*；p4 不含 *，故类型 int。因此题干对四个变量的描述正确。

考场写法

选 a，True。p1、p3 是 int*；p2、p4 是 int。

易错点

把 `int*` 整块机械分发给逗号后所有名字，会错判 p2。空格位置只是排版。初学写代码可一行声明一个变量，阅读题目仍要会拆声明符。

Lecture 定位与这一块还包含什么

U6 第 22 页/张：Declaring a Pointer Variable。还应掌握指针初始化、指针的目标类型、多级指针；注意声明中的 * 与表达式中的 * 含义不同。

2021 Spring Final • Q7 • 取地址与解引用的往返

来源：E21，原卷 PDF 第 2 页；真实卷别 Final。

完整 exam 原题

7. For a variable x, the expression *&x is essentially equivalent to x; and for a pointer p, the expression &*p is equivalent to p.

a. True b. False

详细解析

题目在说什么

原题询问 `*&x` 是否对应 x，以及 `&*p` 是否对应 p。这里不是相乘，也不是把两个字符随意删掉；每个符号有一次独立作用。

第一反应

按一元表达式结构从内向外读：`*(&x)` 和 `&(*p)`，再画访问路径。

核心公式与规则

对题目默认的普通有效对象与内建操作，& 从对象得到地址，* 沿指针访问对象；`*&x` 指认 x，`&*p` 恢复对应指针。

推导

设 x 位于 0x1000，p 保存 0x1000。`&x` 得 0x1000，再解引用回到 x。`*p` 指认位于 0x1000 的 x，对该对象再取地址，得到 0x1000，即 p 的值。因此在本题正常使用条件下两项成立。

考场写法

按原题有效对象指针语境选 a，True。`*(&x)` denotes x, and `&(*p)` yields the corresponding pointer value.

易错点

“可往返”不意味着 p 和 *p 相同，一个是指针访问路径，一个是目标对象。不要将此题扩展为可随意读取无效指针目标，也不要忽略自定义类型可能重载运算符的不同语境。

Lecture 定位与这一块还包含什么

U6 第 23-24、27-29 页/张：指针运算与引用对照。还应掌握修改指针值和修改目标值的区别，以及解引用的有效性条件。

2022 Spring Final • Q7 • 取地址与解引用的往返

来源：E22，原卷 PDF 第 2 页；真实卷别 Final。

完整 exam 原题

7. For a variable x, the expression *&x is essentially equivalent to x; and for a pointer p, the expression &*p is equivalent to p.

a. True b. False

详细解析

题目在说什么

原题询问 `*&x` 是否对应 x，以及 `&*p` 是否对应 p。这里不是相乘，也不是把两个字符随意删掉；每个符号有一次独立作用。

第一反应

按一元表达式结构从内向外读：`*(&x)` 和 `&(*p)`，再画访问路径。

核心公式与规则

对题目默认的普通有效对象与内建操作，& 从对象得到地址，* 沿指针访问对象；`*&x` 指认 x，`&*p` 恢复对应指针。

推导

设 x 位于 0x1000，p 保存 0x1000。`&x` 得 0x1000，再解引用回到 x。`*p` 指认位于 0x1000 的 x，对该对象再取地址，得到 0x1000，即 p 的值。因此在本题正常使用条件下两项成立。

考场写法

按原题有效对象指针语境选 a，True。`*(&x)` denotes x, and `&(*p)` yields the corresponding pointer value.

易错点

“可往返”不意味着 p 和 *p 相同，一个是指针访问路径，一个是目标对象。不要将此题扩展为可随意读取无效指针目标，也不要忽略自定义类型可能重载运算符的不同语境。

Lecture 定位与这一块还包含什么

U6 第 23-24、27-29 页/张：指针运算与引用对照。还应掌握修改指针值和修改目标值的区别，以及解引用的有效性条件。

2021 Spring Final • Q15 • 二级指针先问存的是哪一层地址

来源：E21，原卷 PDF 第 3、4 页；真实卷别 Final。

完整 exam 原题

15. Assuming the following C++ variable declaration statement

```
int i = 2021, *pi = &i, **ppi = &pi;
```

leads to the addresses of i and pi being 0x2020 and 0x2024, respectively, what are the values of ppi and **ppi?

a. 0x2020 and 2021

b. 0x2024 and 2021

c. 0x2020 and 0x2024

d. 0x2024 and 0x2028

详细解析

题目在说什么

声明包含三个对象：i 是 int，pi 是 int*，ppi 是 int。**题目给 i 地址 0x2020、pi 地址 0x2024，问 ppi 的值和** ppi 的值。没有要求 ppi 自身地址。

第一反应

从初始化画链：ppi 保存 &pi，pi 保存 &i，i 保存 2021。先照链找第一项，再沿两次解引用找第二项。

核心公式与规则

`ppi=&pi`，故 ppi 的值等于 pi 对象的地址；`*ppi` 指认 pi；`**ppi` 指认 i。

推导

i 的内容是十进制 2021。pi 内容是 0x2020，但 pi 自己位于 0x2024。ppi 初始化为 &pi，所以 ppi 内容为 0x2024。第一次解引用读取 pi，第二次访问 i，结果 2021。

考场写法

选 b：ppi = 0x2024，**ppi = 2021。

易错点

不要把 ppi 写成 &i=0x2020，那少走了一层。也不要从相邻对象地址差 4 就推断所有机器的指针宽度；本题直接给了所需地址，无须外推。

Lecture 定位与这一块还包含什么

U6 第 23-24、45-46 页/张：指针运算与 int **示例。还应能区分 ppi、*ppi、**ppi、ppi、&ppi 各自类型及含义。

2022 Spring Final • Q15 • 问 *ppi 而不是 ppi

来源：E22，原卷 PDF 第 3、4 页；真实卷别 Final。

完整 exam 原题

15. Assuming the following C++ variable declaration statement

```
int i = 2022, *pi = &i, **ppi = &pi;
```

leads to the addresses of i, pi, and ppi being 0x5190, 0x5198, and 0x51A0, respectively, what are the values of **ppi and *ppi?

a. 0x51A0 and 0x5198

b. 0x5198 and 0x5190

c. 0x5190 and 0x5198

d. 2022 and 0x5190

详细解析

题目在说什么

i 初值 2022，i、pi、ppi 自身地址依次是 0x5190、0x5198、0x51A0。本卷问 **ppi 和 *ppi，顺序也与 2021 卷不同。

第一反应

先写对象内容表，避免把题目给的三个地址直接依次塞进答案。

核心公式与规则

初始化决定 `pi=&i`、`ppi=&pi`；读取 *ppi 得到 pi 这个对象中保存的指针值；再解引用才得到 i 的整数值。

推导

i 位于 0x5190，内容为 2022。pi 位于 0x5198，内容为 0x5190。ppi 位于 0x51A0，内容为 0x5198。因而 *ppi 的值是 0x5190，**ppi 的值是 2022。按题目提问顺序应填 2022 与 0x5190。

考场写法

选 d：**ppi = 2022，*ppi = 0x5190。

易错点

0x5198 是 ppi 的值，不是 *ppi 的值；0x51A0 是 &ppi。只记“二级指针答案是中间地址”无法适应本卷的不同问法。

Lecture 定位与这一块还包含什么

U6 第 45-46 页/张的多级指针例子，基础定义在第 22-24 页。还应掌握每解引用一次，类型减少一层指针，值的角色随之改变。

2021 Spring Final • Q27 • 左值与新算出的指针值

来源：E21，原卷 PDF 第 6 页；真实卷别 Final。

完整 exam 原题

27. In C++, any expression that refers to an internal memory location capable of storing data is called an lvalue, which can appear on the left side of an assignment statement. Which of the following terms in a C++ program cannot be lvalues?

a. p

b. *p

c. &p

d. (p+1)

详细解析

题目在说什么

题目要选不能是左值的表达式。按课程正常语境，p 是普通对象指针，四个表达式都是内建运算。题干把“左值”简化成可以放赋值号左侧，需要补上可修改性的区别。

第一反应

先问它是否指认现有对象，再问是否能修改该对象。不能仅凭表达式中有没有星号或地址值作判断。

核心公式与规则

p 指认指针变量，是左值；*p 指认被指对象，是左值。内建 p+1 产生新的指针值，&p 产生地址值，本例都为纯右值。左值不等于可修改左值。

推导

例如 `p=q` 改指针变量，`*p=7` 改目标对象，因此两者都有明确被写入的对象。`p+1` 是计算出来的地址值，不能对这个计算结果本身赋值；`&p` 也是取地址结果，不能写 `&p=q`。所以要选 &p 与 p+1。两卷选项顺序不同，必须分别对应。

考场写法

选 c、d，即 &p 和 (p+1)。p 与 *p 是左值，但是否可赋值还需检查 const、目标有效性等条件。

易错点

不要把“装地址的指针变量 p”与“计算得到的地址值 p+1”混为一谈。const 对象仍是左值，不能因为不允许赋值就说它不是左值。题干未声明 p，若人为换成有特殊重载的类类型会改变问题，超出本题默认语境。

Lecture 定位与这一块还包含什么

U6 第 18-19、23、37 页/张：lvalue、取地址、不可修改左值。还应对照 const int、数组、指针变量与指针算术结果。

2022 Spring Final • Q26 • 左值与新算出的指针值

来源：E22，原卷 PDF 第 6 页；真实卷别 Final。

完整 exam 原题

26. In C++, any expression that refers to an internal memory location capable of storing data is called an lvalue, which can appear on the left side of an assignment statement. Which of the following terms in a C++ program cannot be lvalues?

a. p

b. (p+1)

c. &p

d. *p

详细解析

题目在说什么

题目要选不能是左值的表达式。按课程正常语境，p 是普通对象指针，四个表达式都是内建运算。题干把“左值”简化成可以放赋值号左侧，需要补上可修改性的区别。

第一反应

先问它是否指认现有对象，再问是否能修改该对象。不能仅凭表达式中有没有星号或地址值作判断。

核心公式与规则

p 指认指针变量，是左值；*p 指认被指对象，是左值。内建 p+1 产生新的指针值，&p 产生地址值，本例都为纯右值。左值不等于可修改左值。

推导

例如 `p=q` 改指针变量，`*p=7` 改目标对象，因此两者都有明确被写入的对象。`p+1` 是计算出来的地址值，不能对这个计算结果本身赋值；`&p` 也是取地址结果，不能写 `&p=q`。所以要选 &p 与 p+1。两卷选项顺序不同，必须分别对应。

考场写法

选 b、c，即 &p 和 (p+1)。p 与 *p 是左值，但是否可赋值还需检查 const、目标有效性等条件。

易错点

不要把“装地址的指针变量 p”与“计算得到的地址值 p+1”混为一谈。const 对象仍是左值，不能因为不允许赋值就说它不是左值。题干未声明 p，若人为换成有特殊重载的类类型会改变问题，超出本题默认语境。

Lecture 定位与这一块还包含什么

U6 第 18-19、23、37 页/张：lvalue、取地址、不可修改左值。还应对照 const int、数组、指针变量与指针算术结果。

2022 Spring Final • Q20 • 函数指针的括号结构

来源：E22，原卷 PDF 第 5 页；真实卷别 Final。

完整 exam 原题

20. What does the following declaration do?

```
double (*f)(double)
```

a. Declares f as a function taking a double and returning a pointer to a double.

b. Declares f as a function object or functor taking a double and returning a double.

c. Declares f as a pointer to a function taking a double and returning a double.

d. Declares *f as a function taking a double and returning a double.

详细解析

题目在说什么

题干给 `double (*f)(double)`，要求说明 f 的声明含义。左边和右边各一个 double，分别承担返回类型和参数类型的作用。

第一反应

从名字 f 向外读。先遇到括号中的 *，说明 f 是指针；再遇到外面的参数括号，说明它指向函数。

核心公式与规则

`R (*p)(A)` 声明指向“参数 A、返回 R”函数的指针；`R *f(A)` 则声明接收 A 并返回 R* 的函数。括号决定 * 与函数声明的绑定结构。

推导

在本题，R 与 A 都是 double，所以 f 指向一个接收一个 double 参数、返回 double 的函数。例如 `double square(double x){return x*x;}` 后可写 `double (*f)(double)=square;`，再通过 `f(3.0)` 得 9.0。f 本身不是函数对象类，也不是返回 double* 的函数。

考场写法

选 c：f is a pointer to a function taking a double and returning a double。完整声明使用时在末尾补分号。

易错点

漏看 (*f) 的括号会误选 a；函数指针与拥有 operator() 的类对象不是同一种类型；最左 double 不表示指针变量自身是 double。

Lecture 定位与这一块还包含什么

U6 第 34 页/张：Pointer to Function。还应掌握回调、把函数作为参数、函数指针类型必须与目标函数匹配，以及指针声明从名字向外读的方法。

06 数组、地址步长与指针自增

知识点先讲

数组把固定数量的同类型元素连续存放。例如 `double a[10]` 是一个包含 10 个 double 的数组对象，每个元素本身也是对象。它解决按下标组织同类数据的需求，但原生数组不会自动检查每次下标是否越界。

allocated size 是已分配的元素槽位数；effective size 是程序当前使用的元素数。若 `int a[4]` 当前保存 2 个有效数据，可以另用 `int used=2` 记录。满时 used 可以等于 4；有效下标仍只能到 3。不能写越界数据，编译器也不一定能阻止一段依赖运行时输入的越界代码。

数组名在许多表达式中会转换为指向首元素的指针。例如 `double* p=a` 让 p 指向 a[0]。转换不等于数组本身就是指针变量；`sizeof(a)` 仍能得到完整数组大小，`&a` 则取得整个数组的地址。

表达式	本例类型/角色	加 1 的步长
`a`（发生转换后）	double*，指向首元素	一个 double
`&a[0]`	double*，指向首元素	一个 double
`&a`	double (*)[10]，指向整个数组	十个 double
`*a`	double 左值，指认首元素	若作 +1，则是数值加法

当题目明确规定 double 占 8 字节，基址 B 为十六进制地址时，`a+i` 的地址数值是 B+8i；`&a+1` 对应 B+80。这里公式是用来手算地址的，不是在 C++ 源码里把指针当整数强行相加。`a+10` 可以表示尾后位置，但不能解引用读取第 11 个元素。

`a[i]` 与 `*(a+i)` 对本例指认同一元素；`*a+i` 则先读 a[0]，再把该数值与 i 相加。指针地址的 + 与 double 的 + 是不同操作，最常见错误就发生在忽略这一步类型判断。

现在才把 ++ 组合进来。先用纯整数理解：`++x` 先使 x 增一，其表达式结果对应增加后的 x；`x++` 产生增加前的值，同时使 x 增一。对指针自增，变化的是指向的元素位置，步长由目标类型决定。

写法	分组	本步得到什么	改什么
`*p++`	`*(p++)`	原 p 指向元素的值	p 后移一个元素
`*++p`	`*(++p)`	p 后移后所指元素的值	p 后移一个元素
`++*p`	`++(*p)`	目标元素增加后的值	改目标元素，p 不动
`(*p)++`	显式括号	目标元素增加前的值	改目标元素，p 不动

手推始终保留三份信息：p 当前指向哪个下标；数组每个元素的当前内容；本条表达式交给左边变量的值。每条语句结束后更新状态，再执行下一条。不能每行都拿数组的初始值重算。

2021 Spring Final • Q5 • 有效长度可以恰好等于容量

来源：E21，原卷 PDF 第 2 页；真实卷别 Final。

完整 exam 原题

5. In a collection class, the effective size can never be equal to or greater than the allocated size.

a. True b. False

详细解析

题目在说什么

题干说集合的 effective size 绝不能等于或大于 allocated size。这里 effective size 是当前元素数，allocated size 是可容纳的槽位数。

第一反应

先用刚好装满的容器作最小反例。不能超过容量，不等于不能达到容量。

核心公式与规则

在正确维护容量的常规模型中，`0 <= effective size <= allocated size`；下标上限则是 size-1。这两个不等式比较的对象不同。

推导

设分配了 4 个槽位，已放入 4 个元素，则 effective=4、allocated=4，状态合法。原题把相等也禁止了，因此即使不讨论动态扩容，题干已经为假。若要再加第 5 个，动态容器需要先扩容，原生数组则不能直接越界写。

考场写法

选 b，False。The effective size may equal the allocated size when the container is full.

易错点

容易把“最后下标小于容量”错套到“元素个数小于容量”。长度 4 的容器最大下标是 3，但可以装 4 个元素。也不要把容量自动增长与原生数组的固定长度混用。

Lecture 定位与这一块还包含什么

U6 第 35-36 页/张：Simple Arrays、allocated/effective size；S10 第 10-15 页：vector 接口。还应区分 size/capacity、下标/长度，以及原生数组和容器扩容。

2022 Spring Final • Q5 • 编译器不会替你维护运行时有效长度

来源：E22，原卷 PDF 第 2 页；真实卷别 Final。

完整 exam 原题

5. In an array, the size of the array specified in the declaration is called the allocated size. The number of elements actively in use is called the effective size. The C++ compiler makes sure that the effective size can never exceed the allocated size.

a. True b. False

详细解析

题目在说什么

本卷先给 allocated size 与 effective size 的定义，再说 C++ 编译器保证后者永远不超过前者。错点是这项自动保证，不是前面的两个名词。

第一反应

检查原生数组有没有每次读写的强制边界检查。一个从 cin 读入下标再写数组的程序可以编译，却可能在运行时越界。

核心公式与规则

合法访问原生数组时要求 `0 <= i < N`。这是程序必须满足的条件，不代表编译器会为每个运行时 i 插入边界检查。越界访问会导致未定义行为。

推导

如 `int a[4]; int i; cin>>i; a[i]=7;`，编译器通常无法预先知道输入是否为 8。若输入 8，写入超出 a 的元素范围。另设的 used 计数变量也可能被程序错误地赋为 5。因此并不存在题干宣称的普遍自动保证。

考场写法

选 b，False。Built-in arrays do not automatically check run-time bounds; the programmer must keep the effective size within capacity.

易错点

“程序必须满足”与“编译器保证满足”是不同命题；不能因为某编译器对常量越界给警告就推断所有越界都能被阻止。越界后的单次运行现象不能当成固定输出。

Lecture 定位与这一块还包含什么

U6 第 35-36 页/张直接说明 array selection does not check range。还应掌握循环上界、传数组时另传长度、std::vector::at 的检查接口与 operator[] 的区别。

2021 Spring Final • Q31 • 逐语句追踪 p 与数组内容

来源：E21，原卷 PDF 第 7、8 页；真实卷别 Final。

完整 exam 原题

31. What is the formatted output of the following program? Write your answer in the box, and make sure the format is correct. (12%)

```
#include <iostream>
using namespace std;
int main(void)
{
    int arr[] = {2, 0, 2, 1};
    int* p = arr;
    int a = *++p;
    int b = *p++;
    int c = ++*p;
    int d = (*p)++;
    cout << arr[0] << arr[1] << arr[2] << arr[3] << endl;
    cout << a << b << c << d << endl;
    return 0;
}
```

Answer box
_____
_____

详细解析

题目在说什么

题目要求原样写出两行输出。初始数组为 {2,0,2,1}，p 指向 arr[0]。a、b、c、d 依次接收四种表达式结果；本卷先输出数组，再输出 a b c d。

第一反应

先在四个表达式外补括号，然后每行记录 p 的下标、数组内容、赋给新变量的值。不要按另一年份的代码顺序推。

核心公式与规则

后缀 ++ 优先于一元 *；前缀 ++ 与一元 * 按其语法结构组合。分别是 `*(++p)`、`*(p++)`、`++(*p)`、`(*p)++`。连续 << 不会自动插入空格。

推导

初始：p→arr[0]，arr={2,0,2,1}。

语句	本条结果	语句后 p	语句后数组
`a=*++p`	先 p→1，再读 0，所以 a=0	1	{2,0,2,1}
`b=*p++`	用原下标 1 读 0，随后 p→2，b=0	2	{2,0,2,1}
`c=++*p`	arr[2] 由 2→3，c=3	2	{2,0,3,1}
`d=(*p)++`	先交出 3，arr[2] 再由 3→4，d=3	2	{2,0,4,1}

第一行按 arr[0] 到 arr[3] 依次输出 2、0、4、1；第二行按 a、b、c、d 输出 0、0、3、3。

考场写法

考场应写两行，行内没有空格：

```
2041
0033
```

易错点

把 *p++ 当成 (*p)++ 会把第二条改成增加元素；忽略上一条修改会误算 c/d；本卷先数组后四个变量，不能照搬 2022 的输出顺序。

Lecture 定位与这一块还包含什么

U1 第 47-48 页/张：前缀与后缀 ++；U6 第 32、37-38、42-43 页/张：优先级、指针步长和混合表达式；U5 第 11-12 页/张：输出。还应能判断越界与语句间状态依赖。

2022 Spring Final • Q31 • 相似代码中的不同执行顺序

来源：E22，原卷 PDF 第 7、8 页；真实卷别 Final。

完整 exam 原题

31. What is the formatted output of the following program? Write your answer in the box, and make sure the format is correct. (12%)

```
#include <iostream>
using namespace std;
int main(void)
{
    int arr[] = {0, 5, 1, 9};
    int* p = arr;
    int a = *++p;
    int b = ++*p;
    int c = *p++;
    int d = (*p)++;
    cout << a << b << c << d << endl;
    cout << arr[0] << arr[1] << arr[2] << arr[3] << endl;
    return 0;
}
```

Answer box
_____
_____

详细解析

题目在说什么

初始 arr={0,5,1,9}，p→arr[0]。四条赋值的顺序是 *++p、++*p、*p++、(*p)++；先打印 a b c d，后打印数组。

第一反应

将指针位置和元素内容分开记录。第二条改 arr[1]，会影响第三条读取的值；第四条已访问 arr[2]。

核心公式与规则

`*++p` 先移指针再读取；`++*p` 先增加所指元素再产生结果；`*p++` 通过原指针读取，再推进指针；`(*p)++` 返回旧元素值并增加元素。

推导

初始：p→0，arr={0,5,1,9}。

语句	本条结果	语句后 p	语句后数组
`a=*++p`	p→1 后读 5，a=5	1	{0,5,1,9}
`b=++*p`	arr[1] 从 5→6，b=6	1	{0,6,1,9}
`c=*p++`	原下标 1 当前值 6，c=6；p→2	2	{0,6,1,9}
`d=(*p)++`	arr[2] 旧值 1 给 d，元素增至 2	2	{0,6,2,9}

第一行拼接 5、6、6、1；第二行拼接 0、6、2、9；endl 各产生一次换行。

考场写法

准确输出：

```
5661
0629
```

易错点

c 不是初始 arr[1] 的 5，因为第二条已将它变成 6；d 不是 2，因为后缀产生旧值；cout 链中没有写空格字面量，所以不能自行加空格。

Lecture 定位与这一块还包含什么

U6 第 42-43 页/张是直接对照；前置是 U1 第 47-48 页/张与 U6 第 37-38 页/张。还应比较把任意一条移动到另一条之前，会如何改变后续状态。

2021 Spring Final • Q32 • 十二个表达式逐格解析

来源：E21，原卷 PDF 第 8 页；真实卷别 Final。

完整 exam 原题

32. Assume that doubleArray is declared as

```
double doubleArray[] = {0, 1, 1, 2, 3, 5, 8, 13, 21, 34};
```

and variables of type double take up 8 bytes on the computer system you are using. If the base address of the array doubleArray happens to be 0xACE0, what are the values of the following 12 expressions? Write your answer in the boxes below, all in 4-character hexadecimal format, e.g., 0 should be 0000. (12 x 1.5% = 18%)

Expression	Answer (four hexadecimal characters)
`doubleArray`	0x ACE0 (pre-filled in original)
`*doubleArray`	0x _____
`&doubleArray`	0x _____
`&doubleArray[0]`	0x _____
`doubleArray[0]`	0x _____
`doubleArray+1`	0x _____
`&doubleArray+1`	0x _____
`&doubleArray[1]`	0x _____
`doubleArray+9`	0x _____
`*doubleArray+9`	0x _____
`*(doubleArray+9)`	0x _____
`&doubleArray[9]`	0x _____
`doubleArray[9]`	0x _____

详细解析

题目在说什么

原表有 13 行：第一行 doubleArray 的基址已填好，只有其后 12 行待填写。下面逐格保留原表顺序；“格 1-12”是讲义添加的定位编号。每格都先判断地址还是数值，再算，最后写四位十六进制。

第一反应

先列类型：数组转换得到 double*；&数组 得到指向整个 double[10] 的指针；解引用或下标则可能得到 double 元素。区分类型后才能选步长或数值加法。

核心公式与规则

每个 double 占 8 字节，整个数组占 10×8=80 字节；a+i 的地址为 B+8i，&a+1 为 B+80；a[i]=*(a+i)。题目要求把这些整数值写成十六进制，不是把 double 的 64 位浮点表示拆成十六进制。

推导

下面每格单独展示语法结构、计算与答案。首行已给出的基址只作已知条件，不重复计入十二个空格。

考场写法

按原表从第二行开始，依次填写：0000、ACE0、ACE0、0000、ACE8、AD30、ACE8、AD28、0009、0022、AD28、0022。每项均保持四位，前面的 0x 由原题印好。

易错点

三个通用检查：& 是否改变了目标类型；* 与 + 谁先结合；算出来的是十进制数值还是十六进制地址。最后检查进位和前导零，不在尾后地址上解引用。

原表待填格 1：`*doubleArray`

1. 题目在说什么。 本格求首元素对象的数值，表达式是 `*doubleArray`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 先沿首元素指针解引用，先确认结果角色再计算。

3. 核心公式。 a[i]=*(a+i)；先解引用再加整数时是普通数值加法，最后做数值的十六进制换算。

4. 推导。 a[0]=0。故本格按四字符补零为 `0000`。

5. 考场写法。 在该行印好的 0x 后填 `0000`。

6. 易错点。 把它误写成数组基址。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 2：`&doubleArray`

1. 题目在说什么。 本格求整个数组的地址，表达式是 `&doubleArray`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 & 的操作数是整个数组，此时不发生首元素指针转换，先确认结果角色再计算。

3. 核心公式。 地址 B+下标×8；若指向完整数组，加一使用 10×8 字节；取现有对象地址则直接读取其位置。

4. 推导。 地址为基址 0xACE0；类型为 double (*)[10]。故本格按四字符补零为 `ACE0`。

5. 考场写法。 在该行印好的 0x 后填 `ACE0`。

6. 易错点。 数值等于首元素地址不表示类型相同。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 3：`&doubleArray[0]`

1. 题目在说什么。 本格求第 0 个元素的地址，表达式是 `&doubleArray[0]`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 先定位下标 0 的元素，再取地址，先确认结果角色再计算。

3. 核心公式。 地址 B+下标×8；若指向完整数组，加一使用 10×8 字节；取现有对象地址则直接读取其位置。

4. 推导。 0xACE0+0×8=0xACE0。故本格按四字符补零为 `ACE0`。

5. 考场写法。 在该行印好的 0x 后填 `ACE0`。

6. 易错点。 这里指向一个 double，不是指向十个 double 的整组。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 4：`doubleArray[0]`

1. 题目在说什么。 本格求下标 0 的元素值，表达式是 `doubleArray[0]`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 下标选择得到对象，再读内容，先确认结果角色再计算。

3. 核心公式。 a[i]=*(a+i)；先解引用再加整数时是普通数值加法，最后做数值的十六进制换算。

4. 推导。 a[0]=0。故本格按四字符补零为 `0000`。

5. 考场写法。 在该行印好的 0x 后填 `0000`。

6. 易错点。 题目问值，不是这个槽位所在地址。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 5：`doubleArray+1`

1. 题目在说什么。 本格求指向第 1 个元素的地址，表达式是 `doubleArray+1`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 首元素指针加一，按单个 double 跨越，先确认结果角色再计算。

3. 核心公式。 地址 B+下标×8；若指向完整数组，加一使用 10×8 字节；取现有对象地址则直接读取其位置。

4. 推导。 0xACE0+8=0xACE8。故本格按四字符补零为 `ACE8`。

5. 考场写法。 在该行印好的 0x 后填 `ACE8`。

6. 易错点。 不能把加一解释成加 1 字节。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 6：`&doubleArray+1`

1. 题目在说什么。 本格求整个数组之后的地址，表达式是 `&doubleArray+1`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 指向数组的指针加一，跨过完整 10 个元素，先确认结果角色再计算。

3. 核心公式。 地址 B+下标×8；若指向完整数组，加一使用 10×8 字节；取现有对象地址则直接读取其位置。

4. 推导。 10×8=80=0x50；0xACE0+0x50=0xAD30。故本格按四字符补零为 `AD30`。

5. 考场写法。 在该行印好的 0x 后填 `AD30`。

6. 易错点。 跨越 80 字节；不是 8 字节，也不是十六进制 0x80。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 7：`&doubleArray[1]`

1. 题目在说什么。 本格求第 1 个元素的地址，表达式是 `&doubleArray[1]`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 下标先定位，再取地址，先确认结果角色再计算。

3. 核心公式。 地址 B+下标×8；若指向完整数组，加一使用 10×8 字节；取现有对象地址则直接读取其位置。

4. 推导。 0xACE0+1×8=0xACE8。故本格按四字符补零为 `ACE8`。

5. 考场写法。 在该行印好的 0x 后填 `ACE8`。

6. 易错点。 & 改变结果角色为地址，不能填元素值 1。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 8：`doubleArray+9`

1. 题目在说什么。 本格求指向第 9 个元素的地址，表达式是 `doubleArray+9`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 首元素指针跨越 9 个 double，先确认结果角色再计算。

3. 核心公式。 地址 B+下标×8；若指向完整数组，加一使用 10×8 字节；取现有对象地址则直接读取其位置。

4. 推导。 9×8=72=0x48；0xACE0+0x48=0xAD28。故本格按四字符补零为 `AD28`。

5. 考场写法。 在该行印好的 0x 后填 `AD28`。

6. 易错点。 十六进制低位 E0+48 有进位，结果不是 ACC8。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 9：`*doubleArray+9`

1. 题目在说什么。 本格求首元素值再加 9，表达式是 `*doubleArray+9`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 一元 * 先结合，所以是 (*a)+9，先确认结果角色再计算。

3. 核心公式。 a[i]=*(a+i)；先解引用再加整数时是普通数值加法，最后做数值的十六进制换算。

4. 推导。 0+9=9，按题目数值记法转十六进制。故本格按四字符补零为 `0009`。

5. 考场写法。 在该行印好的 0x 后填 `0009`。

6. 易错点。 不是先把指针移到 a[9]，因为原题没有那对括号。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 10：`*(doubleArray+9)`

1. 题目在说什么。 本格求第 9 个元素的数值，表达式是 `*(doubleArray+9)`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 括号先移动指针，再解引用，先确认结果角色再计算。

3. 核心公式。 a[i]=*(a+i)；先解引用再加整数时是普通数值加法，最后做数值的十六进制换算。

4. 推导。 a[9]=34；34=2×16+2=0x22。故本格按四字符补零为 `0022`。

5. 考场写法。 在该行印好的 0x 后填 `0022`。

6. 易错点。 34 是十进制；不能原样填 0034。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 11：`&doubleArray[9]`

1. 题目在说什么。 本格求第 9 个元素的地址，表达式是 `&doubleArray[9]`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 对 a[9] 这个对象取地址，先确认结果角色再计算。

3. 核心公式。 地址 B+下标×8；若指向完整数组，加一使用 10×8 字节；取现有对象地址则直接读取其位置。

4. 推导。 0xACE0+72=0xAD28。故本格按四字符补零为 `AD28`。

5. 考场写法。 在该行印好的 0x 后填 `AD28`。

6. 易错点。 它与 a+9 对应同一地址，但与 a[9] 的值 34 不同。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 12：`doubleArray[9]`

1. 题目在说什么。 本格求第 9 个元素的数值，表达式是 `doubleArray[9]`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 下标从 0 开始，所以 9 是第十个元素，先确认结果角色再计算。

3. 核心公式。 a[i]=*(a+i)；先解引用再加整数时是普通数值加法，最后做数值的十六进制换算。

4. 推导。 第十个值为 34=0x22。故本格按四字符补零为 `0022`。

5. 考场写法。 在该行印好的 0x 后填 `0022`。

6. 易错点。 不要取第九个元素 21；索引 9 对应第十项。 下一格仍从原数组声明出发；这些表达式没有修改数组。

Lecture 定位与这一块还包含什么

U6 第 9-12、35-38 页/张：内存地址、数组类型与指针步长；第 3-7 页/张：进制换算。每个下列空格都定位到这些页，分别标出所用规则。还应掌握 sizeof 数组与 sizeof 指针的区别、尾后指针、数组参数退化。

2022 Spring Final • Q32 • 十二个表达式逐格解析

来源：E22，原卷 PDF 第 8 页；真实卷别 Final。

完整 exam 原题

32. Assume that doubleArray is declared as

```
double doubleArray[] = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};
```

and variables of type double take up 8 bytes on the computer system you are using. If the base address of the array doubleArray happens to be 0x5190, what are the values of the following 12 expressions? Write your answer in the boxes below, all in 4 -character hexadecimal format, e.g., 10 should be 000A. (12 x 1.5% = 18%)

Expression	Answer (four hexadecimal characters)
`doubleArray`	0x 5190 (pre-filled in original)
`&doubleArray`	0x _____
`*doubleArray`	0x _____
`doubleArray+1`	0x _____
`&doubleArray+1`	0x _____
`*doubleArray+1`	0x _____
`doubleArray[2]`	0x _____
`&doubleArray[2]`	0x _____
`doubleArray+8`	0x _____
`*doubleArray+8`	0x _____
`*(doubleArray+8)`	0x _____
`doubleArray[9]`	0x _____
`&doubleArray[9]`	0x _____

详细解析

题目在说什么

原表有 13 行：第一行 doubleArray 的基址已填好，只有其后 12 行待填写。下面逐格保留原表顺序； “格 1-12” 是讲义添加的定位编号。每格都先判断地址还是数值，再算，最后写四位十六进制。

第一反应

先列类型：数组转换得到 double*；&数组 得到指向整个 double[10] 的指针；解引用或下标则可能得到 double 元素。区分类型后才能选步长或数值加法。

核心公式与规则

每个 double 占 8 字节，整个数组占 10×8=80 字节；a+i 的地址为 B+8i，&a+1 为 B+80；a[i]=*(a+i)。题目要求把这些整数值写成十六进制，不是把 double 的 64 位浮点表示拆成十六进制。

推导

下面每格单独展示语法结构、计算与答案。首行已给出的基址只作已知条件，不重复计入十二个空格。

考场写法

按原表从第二行开始，依次填写：5190、0001、5198、51E0、0002、0005、51A0、51D0、0009、0011、0013、51D8。每项均保持四位，前面的 0x 由原题印好。

易错点

三个通用检查：& 是否改变了目标类型；* 与 + 谁先结合；算出来的是十进制数值还是十六进制地址。最后检查进位和前导零，不在尾后地址上解引用。

原表待填格 1：`&doubleArray`

1. 题目在说什么。 本格求整个数组的地址，表达式是 `&doubleArray`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 取数组自身地址，保留数组类型，先确认结果角色再计算。

3. 核心公式。 地址 B+下标×8；若指向完整数组，加一使用 10×8 字节；取现有对象地址则直接读取其位置。

4. 推导。 基址 0x5190，类型 double (*)[10]。故本格按四字符补零为 `5190`。

5. 考场写法。 在该行印好的 0x 后填 `5190`。

6. 易错点。 地址相同不代表 &a 和 a 的指针步长相同。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 2：`*doubleArray`

1. 题目在说什么。 本格求首元素数值，表达式是 `*doubleArray`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 先访问 a[0]，再读其值，先确认结果角色再计算。

3. 核心公式。 a[i]=*(a+i)；先解引用再加整数时是普通数值加法，最后做数值的十六进制换算。

4. 推导。 a[0]=1。故本格按四字符补零为 `0001`。

5. 考场写法。 在该行印好的 0x 后填 `0001`。

6. 易错点。 不是数组地址，也不是 double 的底层 IEEE 位模式。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 3：`doubleArray+1`

1. 题目在说什么。 本格求下一个 double 的地址，表达式是 `doubleArray+1`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 a 转成 double*，加一跨 8 字节，先确认结果角色再计算。

3. 核心公式。 地址 B+下标×8；若指向完整数组，加一使用 10×8 字节；取现有对象地址则直接读取其位置。

4. 推导。 0x5190+8=0x5198。故本格按四字符补零为 `5198`。

5. 考场写法。 在该行印好的 0x 后填 `5198`。

6. 易错点。 不能写 5191；指针加法以元素为单位。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 4：`&doubleArray+1`

1. 题目在说什么。 本格求整个数组之后的地址，表达式是 `&doubleArray+1`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 目标类型是 double[10]，跨一个整数组，先确认结果角色再计算。

3. 核心公式。 地址 B+下标×8；若指向完整数组，加一使用 10×8 字节；取现有对象地址则直接读取其位置。

4. 推导。 10×8=80=0x50；0x5190+0x50=0x51E0。故本格按四字符补零为 `51E0`。

5. 考场写法。 在该行印好的 0x 后填 `51E0`。

6. 易错点。 80 是十进制字节数，写成十六进制后是 50。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 5：`*doubleArray+1`

1. 题目在说什么。 本格求首元素数值加一，表达式是 `*doubleArray+1`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 先算 *a，后算普通数值加法，先确认结果角色再计算。

3. 核心公式。 a[i]=*(a+i)；先解引用再加整数时是普通数值加法，最后做数值的十六进制换算。

4. 推导。 1+1=2。故本格按四字符补零为 `0002`。

5. 考场写法。 在该行印好的 0x 后填 `0002`。

6. 易错点。 这条没有先移动指针；要与 *(a+1) 区分。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 6：`doubleArray[2]`

1. 题目在说什么。 本格求下标 2 的元素数值，表达式是 `doubleArray[2]`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 按 0、1、2 依次定位到第三项，先确认结果角色再计算。

3. 核心公式。 a[i]=*(a+i)；先解引用再加整数时是普通数值加法，最后做数值的十六进制换算。

4. 推导。 a[2]=5。故本格按四字符补零为 `0005`。

5. 考场写法。 在该行印好的 0x 后填 `0005`。

6. 易错点。 不要把索引 2 当成第二项 3。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 7：`&doubleArray[2]`

1. 题目在说什么。 本格求下标 2 的元素地址，表达式是 `&doubleArray[2]`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 从基址跨过两个 double，先确认结果角色再计算。

3. 核心公式。 地址 B+下标×8；若指向完整数组，加一使用 10×8 字节；取现有对象地址则直接读取其位置。

4. 推导。 0x5190+2×8=0x5190+0x10=0x51A0。故本格按四字符补零为 `51A0`。

5. 考场写法。 在该行印好的 0x 后填 `51A0`。

6. 易错点。 有 &，答案是位置而非元素 5。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 8：`doubleArray+8`

1. 题目在说什么。 本格求指向下标 8 的地址，表达式是 `doubleArray+8`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 以 double 为单位推进八格，先确认结果角色再计算。

3. 核心公式。 地址 B+下标×8；若指向完整数组，加一使用 10×8 字节；取现有对象地址则直接读取其位置。

4. 推导。 8×8=64=0x40；0x5190+0x40=0x51D0。故本格按四字符补零为 `51D0`。

5. 考场写法。 在该行印好的 0x 后填 `51D0`。

6. 易错点。 两个 8 各有角色：下标偏移与每元素字节数。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 9：`*doubleArray+8`

1. 题目在说什么。 本格求首元素值加 8，表达式是 `*doubleArray+8`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 * 高于 +，先读 a[0]，先确认结果角色再计算。

3. 核心公式。 a[i]=*(a+i)；先解引用再加整数时是普通数值加法，最后做数值的十六进制换算。

4. 推导。 1+8=9。故本格按四字符补零为 `0009`。

5. 考场写法。 在该行印好的 0x 后填 `0009`。

6. 易错点。 不是 a[8]=17；括号位置决定先移动还是先读取。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 10：`*(doubleArray+8)`

1. 题目在说什么。 本格求下标 8 的元素值，表达式是 `*(doubleArray+8)`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 先移动到下标 8，再解引用，先确认结果角色再计算。

3. 核心公式。 a[i]=*(a+i)；先解引用再加整数时是普通数值加法，最后做数值的十六进制换算。

4. 推导。 a[8]=17=0x11。故本格按四字符补零为 `0011`。

5. 考场写法。 在该行印好的 0x 后填 `0011`。

6. 易错点。 17 转成十六进制是 11，不能填 0017。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 11：`doubleArray[9]`

1. 题目在说什么。 本格求下标 9 的元素值，表达式是 `doubleArray[9]`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 直接读第十个元素，先确认结果角色再计算。

3. 核心公式。 a[i]=*(a+i)；先解引用再加整数时是普通数值加法，最后做数值的十六进制换算。

4. 推导。 a[9]=19=0x13。故本格按四字符补零为 `0013`。

5. 考场写法。 在该行印好的 0x 后填 `0013`。

6. 易错点。 不要把十进制 19 原样放进十六进制框。 下一格仍从原数组声明出发；这些表达式没有修改数组。

原表待填格 12：`&doubleArray[9]`

1. 题目在说什么。 本格求下标 9 的元素地址，表达式是 `&doubleArray[9]`。题中 double 占 8 字节，共 10 个元素。

2. 第一反应。 先定位 a[9]，再取其地址，先确认结果角色再计算。

3. 核心公式。 地址 B+下标×8；若指向完整数组，加一使用 10×8 字节；取现有对象地址则直接读取其位置。

4. 推导。 9×8=72=0x48；0x5190+0x48=0x51D8。故本格按四字符补零为 `51D8`。

5. 考场写法。 在该行印好的 0x 后填 `51D8`。

6. 易错点。 地址计算使用 72 字节偏移，不是元素内容 19。 下一格仍从原数组声明出发；这些表达式没有修改数组。

Lecture 定位与这一块还包含什么

U6 第 9-12、35-38 页/张：内存地址、数组类型与指针步长；第 3-7 页/张：进制换算。每个下列空格都定位到这些页，分别标出所用规则。还应掌握 sizeof 数组与 sizeof 指针的区别、尾后指针、数组参数退化。

07 进制与可寻址空间

知识点先讲

二进制、十进制和十六进制是在不同进位规则下写同一个数。十六进制用 0-9、A-F 表示 0-15，一个十六进制位对应四个二进制位。例如 `0x20=2×16+0=32`，`0x11=17`，不是看到 20 就把它当十进制 20。

地址是给可寻址位置编号的值。n 位地址共有 `2^n` 种不同编码。若每个地址对应一个字节，就能编号 `2^n` 个字节。最大的地址编号是 `2^n-1`，但从 0 数到它一共是 `2^n` 个位置。这与长度为 10 的数组最大下标为 9 是同一类“数量与最后编号”的区别。

本卷规定 1KB=2^10 bytes、1MB=2^10 KB、1GB=2^10 MB，所以 1GB=2^30 bytes。按字节寻址的 n 位地址空间换成卷中 GB，是 `2^(n-30)`。这是题设的理论可编号空间，不等于实机已安装内存或操作系统实际允许某个进程使用的空间。

公式必须先检查前提。如果改成一个地址对应两个字节，则容量还要乘 2；不能看到“n 位计算机”就无视题目明确给出的寻址单位。

2021 Spring Final • Q14 • 地址位数换算成字节容量

来源：E21，原卷 PDF 第 3 页；真实卷别 Final。

完整 exam 原题

14. If we have built a 36-bit computer on which the memory address will be represented in one 36- bit long machine word, and the smallest addressable unit is a sequence of 8 consecutive bits called a byte, how much memory can this computer have, theoretically? (Assume 1KB = 2^10 bytes, 1MB = 2^10 KB, 1GB = 2^10 MB.)

a. 36 GB

b. 64 GB

c. 256 GB

d. 2^36 GB

详细解析

题目在说什么

题目明确地址宽度为 36 位，最小寻址单位为一个 8 位字节，并给出二进制的 KB/MB/GB 换算约定。要求理论可管理的内存容量，而非最后地址编号。

第一反应

先算不同地址数 2^36，再乘每地址 1 字节，最后按 2^30 bytes/GB 换单位。

核心公式与规则

容量 = 地址种数 × 每地址单位大小 = 2^36 bytes；1GB=2^30 bytes。因此容量的 GB 数为 2^(36-30)。

推导

36 个二进制位每位有两种取值，共 2^36 个编码。从全零到全一都算一个地址。每个地址对应一字节，所以容量为 2^36 字节。再除以 2^30，得到 2^6=64GB。

考场写法

选 b：2^36/2^30 = 2^6 = 64 GB，使用原题的二进制单位定义。

易错点

最大地址与容量相差一个单位：不要将 2^n-1 当字节数。也不要把“8 位字节”再除以 8 一次；地址数已经在数 byte。题目写 GB 但明确用二进制换算，应服从题设。

Lecture 定位与这一块还包含什么

U6 第 3-12 页/张：进制、Word Size、Address Length and Memory Size。还应区分 bit/byte、容量/最大下标、理论地址空间/实际物理内存。

2022 Spring Final • Q14 • 地址位数换算成字节容量

来源：E22，原卷 PDF 第 3 页；真实卷别 Final。

完整 exam 原题

14. If we have built a 40-bit computer on which the memory address will be represented in one 40- bit long machine word, and the smallest addressable unit is a sequence of 8 consecutive bits called a byte, how much memory can this computer manage, theoretically? (Assume 1KB = 2^10 bytes, 1MB = 2^10 KB, 1GB = 2^10 MB.)

a. 40 GB

b. 256 GB

c. 320 GB

d. 1024 GB

详细解析

题目在说什么

题目明确地址宽度为 40 位，最小寻址单位为一个 8 位字节，并给出二进制的 KB/MB/GB 换算约定。要求理论可管理的内存容量，而非最后地址编号。

第一反应

先算不同地址数 2^40，再乘每地址 1 字节，最后按 2^30 bytes/GB 换单位。

核心公式与规则

容量 = 地址种数 × 每地址单位大小 = 2^40 bytes；1GB=2^30 bytes。因此容量的 GB 数为 2^(40-30)。

推导

40 个二进制位每位有两种取值，共 2^40 个编码。从全零到全一都算一个地址。每个地址对应一字节，所以容量为 2^40 字节。再除以 2^30，得到 2^10=1024GB。

考场写法

选 d：2^40/2^30 = 2^10 = 1024 GB，使用原题的二进制单位定义。

易错点

最大地址与容量相差一个单位：不要将 2^n-1 当字节数。也不要把“8 位字节”再除以 8 一次；地址数已经在数 byte。题目写 GB 但明确用二进制换算，应服从题设。

Lecture 定位与这一块还包含什么

U6 第 3-12 页/张：进制、Word Size、Address Length and Memory Size。还应区分 bit/byte、容量/最大下标、理论地址空间/实际物理内存。

08 生命周期、动态内存与 static

知识点先讲

地址回答“在哪里”，生命周期回答“什么时候还能用”。自动局部对象通常在所在块结束时销毁；不同调用各有自己的局部状态。静态存储期对象持续存在到程序结束附近的清理阶段；动态分配对象的存活则由相应的释放操作或管理它的对象控制。

课程用 static area / stack / heap 三个区域建立运行模型。它便于手画内存图；C++ 语言规范本身更精确地谈存储期与生命周期，并不要求每种实现都有完全一样的物理布局。

```
int* p = new int(7);
*p = 9;
delete p;
p = nullptr;
```

`new int(7)` 创建动态 int 对象并返回指针。p 保存这条访问路径，`*p=9` 修改动态对象。`delete p` 结束该对象生命周期并释放存储；它不会自动清空变量 p，因此示例最后把 p 改为空。`new T[n]` 应与 `delete[]` 配对；一个只声明而没有初始化的指针不会自行分配目标对象。

```
int nextId() {
    static int n = 0;
    return ++n;
}
```

这里 n 只初始化一次，后续调用继续使用同一个 n。连续调用结果为 1、2、3。去掉 static 后，每次调用都重新创建 n 并置 0，结果一直是 1。static 不等于 const：n 能改变，只是它的值跨调用保存。

动态内存最重要的是所有权：谁负责最后释放？普通指针可以建立非拥有的关联，也可以暂时保存拥有的资源地址，类型本身不会自动说明责任。RAII 把释放动作放进管理对象的析构函数；unique_ptr 表达独占管理，shared_ptr 以引用计数共享管理，weak_ptr 观察共享对象而不延长其存活。旧卷中的裸指针用途仍要会读，但写新代码优先让资源管理关系明确。

2021 Spring Final • Q25 • 课程内存区域模型

来源：E21，原卷 PDF 第 6 页；真实卷别 Final。

完整 exam 原题

25. Which of the following are the areas of memory in which values can be stored in a C++ program?

a. Static area

b. Heap

c. Stack

d. Queue

详细解析

题目在说什么

多选题列出 Static area、Heap、Stack、Queue，要求挑出 C++ 程序存储值的内存区域。前三项是课程内存布局的术语；第四项是数据结构。

第一反应

先分类术语：这是问存储区域，不是在列能存数据的抽象容器。队列当然能装元素，但它不是和 stack/heap 并列的程序内存分区。

核心公式与规则

课件模型：静态区域保存静态存储期数据；调用栈模型保存自动局部对象和调用记录；堆模型容纳动态分配数据。具体实现的布局可以不同。

推导

a Static area 对应程序长期存在的静态数据，选。b Heap 对应 new 等动态分配使用的区域，选。c Stack 对应函数调用及自动局部数据的课程模型，选。d Queue 表示 FIFO 队列，队列的对象和元素仍需放到上述某种存储中，因此它不是另一个内存区域。

考场写法

选 a、b、c：Static area、Heap、Stack。Queue is a data structure, not a memory-storage region in this model.

易错点

stack 这个词既可以指调用栈区域，也可以指后进先出的栈容器；题目语境决定含义。不要因为 queue 也能保存值就误选 d。

Lecture 定位与这一块还包含什么

U6 第 13-14 页/张、U7 第 4、18-19 页/张：内存分配与 Heap-Stack Diagrams。还应区分对象的作用域和生命周期、指针变量存在哪里与目标对象存在哪里。

2022 Spring Final • Q12 • 跨函数调用保留局部状态

来源：E22，原卷 PDF 第 3 页；真实卷别 Final。

完整 exam 原题

12. A local variable in a function is usually destroyed when the function returns. If you want to retain its value until the next time the function is called, which keyword should be used before the declaration of the variable:

a. const

b. extern

c. static

d. protected

详细解析

题目在说什么

题目希望函数返回后，某个局部变量的值能保留到下一次调用。候选关键字 const、extern、static、protected 分别涉及不同规则。

第一反应

需求是延长存储期，同时保持局部名字的使用范围。它不要求值永不改变，也不要求跨文件声明。

核心公式与规则

块作用域的 static 局部对象只初始化一次，并在后续调用之间保留状态。名字作用域仍然局限于该块。

推导

以 nextId 为例：第一次到达声明初始化 n=0，++n 后为 1；返回后 n 不销毁。第二次调用不会重新置零，++n 得 2。const 只是限制修改，extern 用于相关声明/链接语义，protected 是类访问控制，都不能表达本题的这种局部持久状态。

考场写法

选 c，static，例如 `static int count=0;`。It preserves the local object and its value across function calls.

易错点

static 不意味着不可修改，所以计数器可递增；局部 static 也不意味着所有其他函数都能直接用这个名字。初始化只一次与函数体只运行一次是两回事。

Lecture 定位与这一块还包含什么

U3 第 34 页/张和 U7 第 24-25 页/张：initRandomSeed 与 static 用途。还应区分局部 static、文件作用域 static、类静态成员这三种位置。

2021 Spring Final • Q26 • 指针连接对象与动态资源

来源：E21，原卷 PDF 第 6 页；真实卷别 Final。

完整 exam 原题

26. Normally in C++ you should not use pointers as much as in the C language, in which pointers are important for the following reasons:

a. Pointers make it possible to reserve new memory during program execution.

b. Pointers allow you to specify the memory addresses for dynamically allocated variables and objects in the heap.

c. Pointers allow you to refer to a large data structure in a compact way.

d. Pointers can be used to record relationships among data items, e.g., linked list.

详细解析

题目在说什么

题目问 C/C++ 指针的重要用途，包含动态存储、紧凑地引用大结构、记录数据间关系。选项次序在两卷中不同，但四项描述相同用途集合。

第一反应

分别想一个实际模型：new 返回地址、把大型对象的地址传给函数、链表结点中的 next。不要把“指针支持这些工作”理解成单独写一个指针声明就完成分配。

核心公式与规则

指针保存访问目标的地址；动态分配由 new 或分配函数完成；一个对象指针的传递通常不复制整个目标结构；结点指针能表达连接关系。

推导

动态分配产生对象，指针记录分配器给出的地址，因此能在程序运行时访问新资源。传一个指针可以在不复制大结构的情况下访问它。链表结点的 next 指针记录后继关系。按课件对“用途”的宽泛表述，四项都在列有效用途；其中 specify addresses 应理解为记录/指认分配到的地址，不是保证程序能任意选择物理地址。

考场写法

按课程用途语境选 a、b、c、d。说明：分配由 new/malloc 等执行，指针负责保存和使用所返回的地址。

易错点

单独 `int* p;` 没有分配一个 int，也没有初始化指针；紧凑引用不等于目标本身内存变小；分配出的地址通常由分配器选择。题目措辞简略，应把机制补齐后再记忆。

Lecture 定位与这一块还包含什么

U6 第 21、26 页/张；U7 第 3、5-9 页/张：指针三类用途与动态分配。关联 S11 第 4-8 页：链表结点通过地址相连。还应掌握所有权、delete 配对、泄漏和悬空指针。

2022 Spring Final • Q25 • 指针连接对象与动态资源

来源：E22，原卷 PDF 第 6 页；真实卷别 Final。

完整 exam 原题

25. Normally in C++ you should not use pointers as much as in the C language, in which pointers are important for the following reasons:

a. Pointers allow you to specify the memory addresses for dynamically allocated variables and objects in the heap.

b. Pointers make it possible to reserve new memory during program execution.

c. Pointers allow you to refer to a large data structure in a compact way.

d. Pointers can be used to record relationships among data items, e.g., linked list.

详细解析

题目在说什么

题目问 C/C++ 指针的重要用途，包含动态存储、紧凑地引用大结构、记录数据间关系。选项次序在两卷中不同，但四项描述相同用途集合。

第一反应

分别想一个实际模型：new 返回地址、把大型对象的地址传给函数、链表结点中的 next。不要把“指针支持这些工作”理解成单独写一个指针声明就完成分配。

核心公式与规则

指针保存访问目标的地址；动态分配由 new 或分配函数完成；一个对象指针的传递通常不复制整个目标结构；结点指针能表达连接关系。

推导

动态分配产生对象，指针记录分配器给出的地址，因此能在程序运行时访问新资源。传一个指针可以在不复制大结构的情况下访问它。链表结点的 next 指针记录后继关系。按课件对“用途”的宽泛表述，四项都在列有效用途；其中 specify addresses 应理解为记录/指认分配到的地址，不是保证程序能任意选择物理地址。

考场写法

按课程用途语境选 a、b、c、d。说明：分配由 new/malloc 等执行，指针负责保存和使用所返回的地址。

易错点

单独 `int* p;` 没有分配一个 int，也没有初始化指针；紧凑引用不等于目标本身内存变小；分配出的地址通常由分配器选择。题目措辞简略，应把机制补齐后再记忆。

Lecture 定位与这一块还包含什么

U6 第 21、26 页/张；U7 第 3、5-9 页/张：指针三类用途与动态分配。关联 S11 第 4-8 页：链表结点通过地址相连。还应掌握所有权、delete 配对、泄漏和悬空指针。

09 类、复制、继承与虚函数

知识点先讲

class 让程序员定义新的类型，把一组数据与操作这些数据的方法放在同一个接口下。对象是这个类型的一个具体实例。class 是类型描述，`Point p;` 中 p 才是对象。struct 在 C++ 里也能有成员函数和构造函数；它与 class 的常见基础区别是默认访问权限。

```
class Counter {
public:
    Counter(int initial) : value(initial) {}
    int get() const { return value; }
    void addOne() { ++value; }
private:
    int value;
};
```

public 成员可作为用户接口；private 成员可由类自身成员及其友元访问；protected 还为派生类提供受规则约束的访问。friend 是授予特定函数/类访问权限的声明，不是第四种访问分区。`Counter(int)` 是构造函数，名字与类相同、没有返回类型；冒号后的初始化列表建立成员初值。`get() const` 约束通过 this 修改当前对象的普通成员。`this` 是当前对象的指针；`Counter::get` 中 `::` 标明函数属于哪个作用域。

复制构造与赋值要分清：`T b=a;` 是创建新的 b 并初始化，通常走复制构造；已有 b 后执行 `b=a;` 是赋值，走 operator=。成员里若有拥有堆资源的原始指针，默认按成员复制会让两个对象保存同一个地址，可能造成共享修改、重复释放。深复制需要另分配目标资源并复制内容；也可以合理地禁止复制，或直接用能正确管理值语义的标准成员。

浅复制与切片是两件事。浅复制问题来自资源所有权；切片则是把派生类对象按值复制成基类对象，后者只能容纳基类部分。自定义复制构造和赋值并不会自动让较小的基类对象装下所有派生类成员。

继承表达类型关系：`class B : public A` 表示 B 公开继承 A；B 对象含有 A 的基类部分。创建最派生对象时，先构造基类，再构造派生类自身；因此基类的初始化结果还可能被后续派生构造函数修改。

虚函数解决“通过基类接口调用，仍选择实际对象对应实现”的需求。先查指针/引用静态类型中选中的函数是否 virtual：是虚函数才走动态分派；否则按静态类型选择。派生覆盖函数无需重复 virtual，仍然是虚函数。反过来，仅在最末层写 virtual，不会把已经存在的基类非虚函数追溯改成虚函数。

```
struct Base { virtual int value() { return 1; } };
struct Derived : Base { int value() override { return 2; } };
Derived d;
Base* p = &d;
// p->value() 得到 2，因为 Base 中的 value 是 virtual
```

`p->value()` 等价于通过 `(*p).value()` 访问成员。函数选择和成员值计算要分两步：先确认调用哪个函数，再用对象当前状态执行那个函数体。下面两年的 Q30 正好是最小对照。

2021 Spring Final • Q12 • 推荐的数据成员访问权限

来源：E21，原卷 PDF 第 3 页；真实卷别 Final。

完整 exam 原题

12. In the definition of a class, it is preferred that the data members be declared as:

a. public

b. protected

c. private

d. friend

详细解析

题目在说什么

题干问设计类时数据成员 preferred 被声明为什么，是设计偏好题；不是问其他访问标签是否语法合法。

第一反应

想清封装目的：把内部表示藏在实现里，通过 public 方法维持对象规则。

核心公式与规则

private 数据成员阻止普通外部代码直接访问；public 方法提供受控接口。protected 主要涉及派生类，friend 不是数据成员访问区标签。

推导

若坐标、容量或计数可以被任意外部语句改写，类难以保证内部关系。将这些字段设为 private，允许外部通过构造函数、getter 和必要的修改方法操作，更便于维护不变条件。因此课程推荐 private，对应 c。

考场写法

选 c，private。Expose appropriate operations through the public interface and keep the representation private.

易错点

public 数据成员并非一概非法，所以不能把推荐写成语言禁止。protected 与 private 的选择涉及继承需求，但本题问一般封装实践；friend 也不是 fourth access specifier。

Lecture 定位与这一块还包含什么

S08 第 15-18、58 页：封装、类定义格式与 getter/setter；U4 第 19-20 页/张作前置。还应掌握接口/实现、默认访问权限和必要时才提供 setter。

2022 Spring Final • Q24 • 合法访问分区与 friend 的区别

来源：E22，原卷 PDF 第 6 页；真实卷别 Final。

完整 exam 原题

24. In the definition of a class, the data members can be declared as:

a. public

b. protected

c. friend

d. private

详细解析

题目在说什么

本题多选问类的数据成员可以按哪些访问权限声明。与 2021 Q12 问“推荐用哪个”不同，这里询问可用访问类别。

第一反应

把 public/protected/private 三个访问分区与 friend 授权机制分开。

核心公式与规则

类访问说明符有 public、protected、private。friend 声明特定外部函数或类为友元，不是允许把普通字段声明为 friend 的第四类标签。

推导

a public 可以用于数据成员；b protected 可以用于数据成员；d private 也可以。c friend 能授予外部函数或类访问权限，但不能像 `public:` 一样作为字段访问分区。因此选 a、b、d。

考场写法

选 a、b、d。The access specifiers are public, protected and private; friend grants access to a designated function or class.

易错点

不要照搬上一年的 private 单选答案；“可以”与“通常推荐”不同。friend 函数可能访问 private 成员，不代表那个数据成员本身成为 friend 类型。

Lecture 定位与这一块还包含什么

S08 第 15-16、39、53、61 页：访问分区、friend 和继承访问关系。还应区分成员访问权限与 public/protected/private 继承方式。

2021 Spring Final • Q21 • OOP 的识别特征与抽象概念

来源：E21，原卷 PDF 第 5 页；真实卷别 Final。

完整 exam 原题

21. What are the distinguishing features of object-oriented programming languages like C++ (i.e., what makes object-oriented programming different from other programming paradigms)?

a. Abstraction

b. Encapsulation

c. Inheritance

d. Polymorphism

详细解析

题目在说什么

题目强调 distinguishing features，并用括号解释“使 OOP 与其他范式有区别的特征”。四项是抽象、封装、继承、多态。它比“列举 OOP 中能使用的概念”更强调区分作用。

第一反应

先把每个词转成具体机制：抽象关注接口与必要信息；封装组织并限制对象内部访问；继承表达类型关系；多态让同一接口对应不同实现。

核心公式与规则

本讲采用 S08 第 58-60 页依次展开的封装、 多态、继承三项作为 OOP 识别特征。抽象也用于函数、模块和其他范式，因此不宜仅因它出现在 OOP 教材中就说它是 OOP 的专属区别。

推导

b 对应把数据与相关行为封装进类；c 对应基类/派生类层次；d 对应通过共同接口取得不同类型的行为。a 抽象本身更一般：普通过程函数也可以隐藏算法细节并给出接口。依据题目“区别于其他范式”的限定，推荐 b、c、d。若课程另用“四大支柱”口径把 abstraction 一并列入，可能给四项；现有资料没有这两卷的官方答案，不能将某份批注当作裁决。

考场写法

按 distinguishing features 的限定推荐 b、c、d，并能解释 abstraction 是跨范式概念。本题有术语口径歧义，不把这一推荐冒充已核对的官方答案。

易错点

不要只背英文词清单； “某概念用于 OOP”不等于“它只属于 OOP” 。也不要因为题面相同就把批注里的勾选当成官方解答；关键是说清术语口径。

Lecture 定位与这一块还包含什么

S08 第 13、58-61 页：Classes and Objects、Encapsulation、Polymorphism、Inheritance；S10 第 5-9 页另谈 ADT 抽象。还应掌握函数抽象与数据抽象、接口和实现，以及编译期/运行期多态的区别。

2022 Spring Final • Q21 • OOP 的识别特征与抽象概念

来源：E22，原卷 PDF 第 5 页；真实卷别 Final。

完整 exam 原题

21. What are the distinguishing features of object-oriented programming languages like C++ (i.e., what makes object-oriented programming different from other programming paradigms)?

a. Abstraction

b. Encapsulation

c. Inheritance

d. Polymorphism

详细解析

题目在说什么

题目强调 distinguishing features，并用括号解释“使 OOP 与其他范式有区别的特征” 。四项是抽象、封装、继承、多态。它比“列举 OOP 中能使用的概念”更强调区分作用。

第一反应

先把每个词转成具体机制：抽象关注接口与必要信息；封装组织并限制对象内部访问；继承表达类型关系；多态让同一接口对应不同实现。

核心公式与规则

本讲采用 S08 第 58-60 页依次展开的封装、 多态、继承三项作为 OOP 识别特征。抽象也用于函数、模块和其他范式，因此不宜仅因它出现在 OOP 教材中就说它是 OOP 的专属区别。

推导

b 对应把数据与相关行为封装进类；c 对应基类/派生类层次；d 对应通过共同接口取得不同类型的行为。a 抽象本身更一般：普通过程函数也可以隐藏算法细节并给出接口。依据题目“区别于其他范式”的限定，推荐 b、c、d。若课程另用“四大支柱”口径把 abstraction 一并列入，可能给四项；现有资料没有这两卷的官方答案，不能将某份批注当作裁决。

考场写法

按 distinguishing features 的限定推荐 b、c、d，并能解释 abstraction 是跨范式概念。本题有术语口径歧义，不把这一推荐冒充已核对的官方答案。

易错点

不要只背英文词清单；“某概念用于 OOP”不等于“它只属于 OOP”。也不要因为题面相同就把批注里的勾选当成官方解答；关键是说清术语口径。

Lecture 定位与这一块还包含什么

S08 第 13、58-61 页：Classes and Objects、Encapsulation、Polymorphism、Inheritance；S10 第 5-9 页另谈 ADT 抽象。还应掌握函数抽象与数据抽象、接口和实现，以及编译期/运行期多态的区别。

2022 Spring Final • Q29 • 复制控制不能自动消除切片

来源：E22，原卷 PDF 第 7 页；真实卷别 Final。

完整 exam 原题

29. To avoid the default shallow copy or slicing in C++, which of the following functions should be overloaded when designing new classes:

a. The copy constructor

b. The assignment operator (=)

c. The insertion operator (<<)

d. The selection operator ([])

详细解析

题目在说什么

题干把 shallow copy 与 slicing 放在一起，问定义新类时应控制哪些函数。四项包括复制构造、赋值、输出和下标运算符。

第一反应

先定位创建副本和替换已有对象这两条复制入口：复制构造函数与复制赋值运算符。输出、下标不承担对象整体复制职责。

核心公式与规则

复制构造常见签名 `T(const T& src)`；复制赋值常见签名 `T& operator=(const T& src)`。拥有原始资源时要按设计完成深复制或禁止复制。切片来自按值形成基类对象，不能仅靠重载这两项普遍消除。

推导

a 在新对象初始化时控制如何复制资源；b 在已有对象赋值时控制资源替换。c 的 << 只定义输出，d 的 [] 只定义元素选择，不直接解决复制所有权问题。因而课程预期选 a、b。需要修正题干的暗示：如 `Base b=derived;`，结果的类型仍是 Base，增加复制函数不会自动保存 derived 的全部派生部分；避免该切片通常需保留基类引用/指针，或设计多态克隆等接口。

考场写法

选 a、b，针对复制控制。补充：它们可用于正确资源复制，但不能一般性地消除派生到基类的按值切片。

易错点

不要把“复制指针值”当成“复制其指向的数据”。也不要认为所有类都要手写复制函数：标准值成员可让默认复制正确工作。课程 S08 第 28 页的 deepCopy 样例有重复释放/初始化检查风险，不能原样作为可运行安全模板照抄。

Lecture 定位与这一块还包含什么

S08 第 21-25、27-28 页：Copying Objects 与深复制；第 60-61 页：继承。还应掌握析构/复制控制的配合、自赋值、所有权与 Rule of Zero 的目的。

2021 Spring Final • Q30 • 基类非虚函数按静态类型调用

来源：E21，原卷 PDF 第 7 页；真实卷别 Final。

完整 exam 原题

30. After running the following program, which of the numbers will be seen in the output?

```
#include <iostream>
using namespace std;
class A {
public:
    int a;
    A() { a = 1; }
    void display() { cout << a << endl; }
};
class B: public A {
public:
    int b;
    B() { b = 2; }
    void display() { cout << a << b << endl; }
};
class C: public B {
public:
    int c;
    C() { b = 1; c = 3; }
    virtual void display() { cout << a << b << c << endl; }
};
int main() {
    C oC;
    B* pb = &oC;
    pb->display();
    return 0;
}
```

a. 1

b. 2

c. 3

d. None, compile error.

详细解析

题目在说什么

A、B、C 逐层继承。main 创建 C 对象，用 B* 指向它，再执行 pb->display()。题目问输出里出现哪些数字，不是问对象有几个成员。

第一反应

先走 A→B→C 的构造顺序算最终成员值，再从 B* 的静态类型检查 B::display 是否 virtual。不能只在整段代码里搜到一个 virtual 就判动态分派。

核心公式与规则

基类先构造，派生类构造后执行。通过指针调用非虚成员函数时，按静态类型选中函数；在派生 C 中新写 virtual 不会追溯改变 B 中原有的非虚函数。

推导

A 构造使 a=1；B 构造使 b=2；C 构造再把 b 改为 1，并设置 c=3。最终对象状态为 a=1、b=1、c=3。

pb 的静态类型是 B*。A::display 与 B::display 都没有 virtual，故 `pb->display()` 调用 B::display。该函数只执行 `cout << a << b << endl`，读取当前对象的 a=1、b=1，输出 11 后换行。C::display 不会被本次调用选中。

考场写法

选 a（数字 1）。实际输出是：

```
11
```

易错点

C 的 virtual 不会让所有同名基类函数事后都变成虚函数。B 构造时 b=2 不代表最后仍为 2；C 已覆盖该值。`11` 包含的是两个数字 1，不应把它看成选项里没有的“十一”从而选编译错误。

Lecture 定位与这一块还包含什么

S08 第 20、41-42、59-61 页：构造顺序所需前置、虚函数与继承；U6 第 30-31 页/张：对象指针和 ->。还应区分静态类型、动态类型、重载与覆盖。

2022 Spring Final • Q30 • 虚函数覆盖继承到后续派生类

来源：E22，原卷 PDF 第 7 页；真实卷别 Final。

完整 exam 原题

30. After running the following program, which of the numbers will be seen in the output?

```
#include <iostream>
using namespace std;
class A {
public:
    int a;
    A() { a = 1; }
    virtual void display() { cout << a << endl; }
};
class B: public A {
public:
    int b;
    B() { b = 2; }
    void display() { cout << a << b << endl; }
};
class C: public B {
public:
    int c;
    C() { a = 2; c = 3; }
    void display() { cout << a << b << c << endl; }
};
int main() {
    C oC;
    B* pb = &oC;
    pb->display();
    return 0;
}
```

a. 1

b. 2

c. 3

d. None, compile error.

详细解析

题目在说什么

本卷 A::display 一开始就带 virtual；C 的构造函数把 a 改为 2、c 设为 3。仍然以 B* 指向 C 对象，要求判断哪些数字出现在输出中。

第一反应

构造状态与函数分派分别判断。A 声明为 virtual 后，B 和 C 的同签名覆盖即使省略 virtual，也继续参与虚调用。

核心公式与规则

公开派生对象可以由合适的基类指针指向；调用选中的成员是虚函数时，使用实际对象的最终覆盖函数。本题 C 对象的最终覆盖是 C::display。

推导

A 构造：a=1。B 构造：b=2。C 构造：a=2、c=3。最终状态为 a=2、b=2、c=3。

pb 虽是 B*，但其目标实际为 C，而且 display 沿继承链始终是虚函数。因此调用 C::display，按顺序输出 a、b、c，得到 223 后换行。没有数字 1，也不发生编译错误。

考场写法

选 b、c（数字 2 和 3）。实际输出：

```
223
```

易错点

不要因为 C 中没再次写 virtual 就认为不覆盖。也不要认为成员 a 属于 A 就永远保持 A 构造写入的 1；它是同一个基类子对象中的可修改成员。

Lecture 定位与这一块还包含什么

S08 第 41-42、59-61 页是直接依据；构造函数在第 20 页。还应掌握 override 用于检查覆盖、通过基类引用调用与基类指针调用，以及复制成基类对象造成的切片。

10 模板、迭代器与基础容器接口

知识点先讲

本章把前面的类型与指针模型扩展到容器。L9 的模板/迭代器属于 C++ 主线；L10-L11 的基础容器作为本次暂定边界补充，不能据此断言教师已经确认期中考到第 11 讲。

模板让同一套代码按类型复用。`template<typename T>` 引入类型参数 T，不是在程序运行时建立名叫 T 的变量。`std::vector<int>` 与 `std::vector<double>` 把不同元素类型代入容器模板。

```
template<typename T>
T larger(T a, T b) { return a < b ? b : a; }
```

模板定义放在前面后，`larger(2,5)` 可推导 T 为 int。它要求实际类型支持所用比较和结果复制等操作；模板不是无条件适配任意类型。函数模板、普通重载与运行期虚函数多态是不同机制。

vector 提供可变长序列与下标访问；stack 限定主要从一端按后进先出处理；queue 按先进先出处理；list 用链式关系组织元素。容器对外暴露哪些操作，比“内部其实能怎样存”更直接决定使用方式。

迭代器提供“当前位置、读取、前进、结束判断”的接口。它借用了指针语法，但未必就是原始指针。对于一个可迭代容器 c：

```
for (auto it = c.begin(); it != c.end(); ++it) {
    std::cout << *it;
}
```

`begin()` 给首元素位置，`end()` 给末元素之后的终止位置；`*it` 读取当前位置，`++it` 前进。循环条件先确认 it 不等于 end，才进入循环解引用。空容器满足 begin==end，循环直接跳过。end 本身不是可以读取的末元素。

range-based for 把这套过程简写为 `for (auto x : c)`。它需要能取得起点、终点并进行迭代的相应接口。std::vector、std::map 提供这些接口；标准库 std::stack 与 std::queue 是容器适配器，通常只公开指定出入操作，不提供 begin/end。因此不能根据内部存储容器有迭代器，就给它们直接写范围 for。

必须保留库版本语境：旧卷大写的 Stanford `Stack/Queue/Vector/Map` 与标准库小写类型不是同一套类。某些教学库或自写 Stack 可以公开迭代器；不能把“栈”这个抽象结构本身说成永远不能遍历。S09 的自定义 Stack 迭代示例恰好说明接口可以由作者设计。

2021 Spring Final • Q9 • end 是末尾之后的位置

来源：E21，原卷 PDF 第 2 页；真实卷别 Final。

完整 exam 原题

9. If c is a nonempty collection in C++ STL, calling c.begin() and c.end() will return iterators pointing at the first and last element of that collection, respectively.

a. True b. False

详细解析

题目在说什么

题目假设 c 非空，称 begin 指向第一个元素、end 指向最后一个元素。需要分别核对两个接口的含义。

第一反应

想一个三元素容器，把 begin、最后元素、end 画成三个不同位置；再对照典型循环条件 it!=end。

核心公式与规则

常规标准容器的迭代区间为 [begin,end)：包含 begin，不包含 end。end 是 past-the-end 位置，不能当元素解引用。

推导

对 vector {10,20,30}，begin 读到 10；最后元素迭代器读到 30；end 则在 30 后面。遍历到最后元素时仍满足 it!=end，处理完再 ++ 到 end，此时停止。若 end 已指向最后元素，这种循环会提前漏掉 30，所以题干后半句错误。

考场写法

选 b，False。begin points to the first element, while end denotes the position one past the last element.

易错点

非空条件不使 end 变成有效元素；空容器则 begin==end。不要给所有迭代器都直接写 end-1：只有支持相关运算的迭代器才允许，不能从 vector 推广到所有容器。

Lecture 定位与这一块还包含什么

S09 第 26-30 页：Using Iterators，特别是第 28 页；S10 第 24-29 页：正向/反向迭代。还应掌握半开区间、空容器、auto 与 auto& 的元素复制差别，以及迭代器失效。

2022 Spring Final • Q9 • end 是末尾之后的位置

来源：E22，原卷 PDF 第 2 页；真实卷别 Final。

完整 exam 原题

9. If c is a nonempty collection in C++ STL , calling c.begin() and c.end() will return iterators pointing at the first and last element of that collection, respectively.

a. True b. False

详细解析

题目在说什么

题目假设 c 非空，称 begin 指向第一个元素、end 指向最后一个元素。需要分别核对两个接口的含义。

第一反应

想一个三元素容器，把 begin、最后元素、end 画成三个不同位置；再对照典型循环条件 it!=end。

核心公式与规则

常规标准容器的迭代区间为 [begin,end)：包含 begin，不包含 end。end 是 past-the-end 位置，不能当元素解引用。

推导

对 vector {10,20,30}，begin 读到 10；最后元素迭代器读到 30；end 则在 30 后面。遍历到最后元素时仍满足 it!=end，处理完再 ++ 到 end，此时停止。若 end 已指向最后元素，这种循环会提前漏掉 30，所以题干后半句错误。

考场写法

选 b，False。begin points to the first element, while end denotes the position one past the last element.

易错点

非空条件不使 end 变成有效元素；空容器则 begin==end。不要给所有迭代器都直接写 end-1：只有支持相关运算的迭代器才允许，不能从 vector 推广到所有容器。

Lecture 定位与这一块还包含什么

S09 第 26-30 页：Using Iterators，特别是第 28 页；S10 第 24-29 页：正向/反向迭代。还应掌握半开区间、空容器、auto 与 auto& 的元素复制差别，以及迭代器失效。

2021 Spring Final • Q23 • 先辨认 Stanford 库版本

来源：E21，原卷 PDF 第 5、6 页；真实卷别 Final。

完整 exam 原题

23. Which of the following collection classes in the Stanford C++ Library do not support the use of the range-based for loop?

a. Vector

b. Stack

c. Queue

d. Map

详细解析

题目在说什么

原题明确写 Stanford C++ Library，选项为大写 Vector、Stack、Queue、Map。它问哪些类不支持 range-based for，而不是问哪些抽象结构无法逐个查看元素。

第一反应

先看题面指定的库，不能直接把它替换为 std 库。再核查该教学版本是否向客户公开 begin/end 和迭代器。

核心公式与规则

range-based for 依赖可用的范围/迭代接口。旧版 Stanford Stack 与 Queue 只公开栈顶或队首等受限操作；Vector、Map 对外提供迭代支持。其他新版或自定义扩展可能不同。

推导

按本卷所处的旧版 Stanford 接口，Vector 可以遍历元素，Map 可以遍历键；Stack 与 Queue 不提供供这种范围循环使用的公开迭代接口。因此课程预期选 b、c。已核对 Stanford 作者维护的旧版 stack.h/queue.h 文档作为版本依据；新课件中自定义的 Stack::iterator 示例不能倒推成这份旧卷也使用该实现。

考场写法

按旧版 Stanford 库语境选 b、c：Stack、Queue。判断依据是该版本公开的接口，而非“栈/队列天生不能迭代”。

易错点

把 Map 迭代得到的键与 std::map 迭代得到的键值 pair 混用会读错循环体。教学库接口可随版本变化，不能只按同名类推断；这也是本题的范围边界说明。

Lecture 定位与这一块还包含什么

S09 第 26-30 页讲通用迭代机制，S10 第 17-20 页、S11 第 26-29 页讲栈/队列接口；这些是当前对应知识，不是旧 Stanford API 的逐字复刻。旧库接口另参见附录的 Stanford 原始头文件链接。

2022 Spring Final • Q23 • 标准适配器不公开 begin/end

来源：E22，原卷 PDF 第 5、6 页；真实卷别 Final。

完整 exam 原题

23. Which of the following collection classes do not support the use of the range-based for loop?

a. vector

b. stack

c. queue

d. map

详细解析

题目在说什么

本卷明确写 C++ STL，选项改为小写 vector、stack、queue、map，问不支持范围 for 的类。需按标准库接口判断。

第一反应

逐个问：是否对对象公开了范围 for 所需的起止迭代接口？底层实现使用何种容器不是直接答案。

核心公式与规则

std::vector 与 std::map 提供 begin/end；std::stack、std::queue 是容器适配器，不公开这对迭代接口。范围 for 不能直接遍历后两者。

推导

a vector 可依次访问其元素；d map 可依次访问键值 pair。b stack 通常只能 top/push/pop，c queue 通常只能 front/back/push/pop。即使 stack/queue 内部使用 deque，该成员是内部实现，不能作为对象自身的公开 begin/end。因此选 b、c。

考场写法

选 b、c：std::stack 与 std::queue 不直接支持 range-based for；std::vector 与 std::map 支持。

易错点

不要将“底层有迭代器”当作“外层适配器也有”。也不要把 S09 自定义 Stack 示例当作 std::stack API；它们名字相近、接口可以不同。

Lecture 定位与这一块还包含什么

S09 第 26-30 页，S10 第 19 页，S11 第 28 页。课件第 28 页“Every collection class in STL”表述应限定到相应容器，不能包括不暴露迭代器的所有适配器。还应练习 front/top 与 pop 的区别：标准 pop 通常返回 void。

11 边界补充：位运算提取颜色分量

知识点先讲

本章沿用旧 M10 的查漏目标，完整解析两卷 Q17，但明确标作边界补充：用户版 U6 讲进制；系统的位运算说明实际位于同教师 2026 春第 14 讲 Sets，第 18-29 页，超出本次暂定 L1-L11 主体。因此本章不作为已确认期中考点。

先补够理解题目的规则。一个十六进制位占 4 个二进制位，两个十六进制位合成一字节。题目把 32 位数按高位到低位分成四字节：alpha、Red、Green、Blue。例如 `0x05 19 20 22` 的四个分量值分别由 05、19、20、22 表示。

右移 `>> 8` 将位模式向低位方向移一字节；原来的 Green 字节会落到最低一字节。`& 0xFF` 是按位与掩码：FF 的低八位全为 1，其余位为 0，所以保留最低八位、清除更高位。这里 & 是二元位运算，既不是声明引用，也不是一元取地址。

```
unsigned int pixel = 0x05192022u;
unsigned int green = (pixel >> 8) & 0xFFu;
```

这段教学变式刻意使用无符号类型。原题代码仍完整保留；讨论实际移位时要考虑有符号整数和语言版本。输出答案前把保留下来的两位十六进制转为十进制，例如 0x20=32、0x21=33。

2021 Spring Final • Q17 • 0x21 的十进制值是 33

来源：E21，原卷 PDF 第 4 页；真实卷别 Final。

完整 exam 原题

17. Computers typically represent each pixel in an image as a 32-bit integer, in which the 4 bytes are interpreted as the α (Transparency), Red, Green, and Blue components. The following bitwise operations extracts one of the components from the hexadecimal representation of the αRGB color stored in an integer variable pixel. Which color is extracted and what is the component value in decimal format?

```
int pixel = 0xFF202104;
int color = (pixel >> 8) & 0xFF;
```

a. α, 0

b. α, 255

c. Green, 21

d. Green, 33

详细解析

题目在说什么

像素按 alpha、Red、Green、Blue 四字节打包，代码为 `(pixel >> 8) & 0xFF`，pixel 初始位模式 0xFF202104。问提取哪个分量及其十进制值。

第一反应

先把八位十六进制数按两位一组拆开：FF | 20 | 21 | 04。再看右移 8 位会让哪一组进入最低字节。

核心公式与规则

一个字节等于两个十六进制位；右移一字节后与 0xFF，只保留原数的第 8-15 位。按题设 alphaRGB 布局，这组是 Green。

推导

四组分别是 alpha=FF、Red=20、Green=21、Blue=04。右移后，原 21 落到低字节，再 &FF 得 0x21。十进制换算为 2×16+1=33。原代码把 0xFF202104 存入 int：在常用 32 位实现/C++17 语境涉及实现定义的有符号转换与右移细节；课程按 32 位像素位模式解释，得到本答案。稳妥实际代码宜使用 uint32_t 等无符号类型。

考场写法

按题设 32 位像素模型选 d：Green，33。`(pixel >> 8) & 0xFF` 保留的是 Green 字节 0x21。

易错点

选 Green,21 是忘了从十六进制换算。最高字节 FF 是透明度，但右移 8 再掩码不会保留它。不要将原卷的 int 写法推广成跨所有旧标准实现都无需限定的位操作范式。

Lecture 定位与这一块还包含什么

U6 第 3-7 页/张提供进制前置；S14《14-CSC3002_Lecture Notes Sets.pdf》原 PDF 第 18-20、27-29 页提供位与/移位。仓库附此相关页摘录并保留原页码对应，明确不并入期中主体。

2022 Spring Final • Q17 • 先移位，再掩码，最后换进制

来源：E22，原卷 PDF 第 4 页；真实卷别 Final。

完整 exam 原题

17. Computers typically represent each pixel in an image as a 32-bit integer or 8 hexadecimal digits (starting with 0x), in which the 4 bytes are interpreted as the transparency (α) and color (Red, Green, Blue) components. The following bitwise operations extract one of the components from the hexadecimal representation of the αRGB color stored in an integer variable pixel. Which component is extracted and what is the value of that component in decimal format?

```
int pixel = 0x05192022;
int color = (pixel >> 8) & 0xFF;
```

a. α, 0

b. α, 256

c. Green, 20

d. Green, 32

详细解析

题目在说什么

本卷 pixel 改为 0x05192022，布局仍是 alphaRGB。要判断右移 8 位再与 0xFF 后留下哪一字节，并用十进制报告。

第一反应

拆成 05 | 19 | 20 | 22，给各组标上 alpha、R、G、B。只追踪 Green 那组的位置移动。

核心公式与规则

右移 8 位相当于把字节组往右挪一格；位与 0xFF 保留最低八位。两个运算共同抽取原来的 Green 字节。

推导

pixel 为正的 0x05192022，右移后为 0x00051920；再与 0x000000FF 得 0x00000020。0x20=2×16=32，因此分量是 Green、数值 32。这里 0x20 是结果的十六进制写法，不是十进制 20。

考场写法

选 d：Green，32。`0x05192022 >> 8 = 0x00051920`，再 &0xFF 得 0x20，即 32。

易错点

&0xFF 是清掉高位而非取高位；右移的数量单位是 bit，8 是一字节。操作数是整数时的 >> 不要与输入流 cin>> 的重载含义混淆。

Lecture 定位与这一块还包含什么

定位同上一题：U6 第 3-7 页/张；S14 原 PDF 第 18-20、27-29 页。还应掌握位或、异或、按位非与逻辑 &&/|| 的区别；这些扩展知识的完整课件在期中主体之外。

原题覆盖索引与未纳入范围

48 道是按原卷题号计数。Q32 的格编号是教学定位，不伪装成原卷小问编号。第 11 专题为边界补充，其余为暂定 L1-L11 内容对应。

原卷	题号	原 PDF 页	专题
2021 Spring Final	Q1	2	05 对象、地址、指针与间接访问
2021 Spring Final	Q2	2	02 函数、参数、引用、const 与重载
2021 Spring Final	Q3	2	03 字符、C 字符串与 std::string
2021 Spring Final	Q4	2	04 流对象、引用返回与连续输出
2021 Spring Final	Q5	2	06 数组、地址步长与指针自增
2021 Spring Final	Q6	2	05 对象、地址、指针与间接访问
2021 Spring Final	Q7	2	05 对象、地址、指针与间接访问
2021 Spring Final	Q9	2	10 模板、迭代器与基础容器接口
2021 Spring Final	Q11	3	01 从类型出发读表达式
2021 Spring Final	Q12	3	09 类、复制、继承与虚函数
2021 Spring Final	Q14	3	07 进制与可寻址空间
2021 Spring Final	Q15	3、4	05 对象、地址、指针与间接访问
2021 Spring Final	Q17	4	11 边界补充：位运算提取颜色分量
2021 Spring Final	Q20	5	02 函数、参数、引用、const 与重载
2021 Spring Final	Q21	5	09 类、复制、继承与虚函数
2021 Spring Final	Q22	5	02 函数、参数、引用、const 与重载
2021 Spring Final	Q23	5、6	10 模板、迭代器与基础容器接口

原卷	题号	原 PDF 页	专题
2021 Spring Final	Q24	6	01 从类型出发读表达式
2021 Spring Final	Q25	6	08 生命周期、动态内存与 static
2021 Spring Final	Q26	6	08 生命周期、动态内存与 static
2021 Spring Final	Q27	6	05 对象、地址、指针与间接访问
2021 Spring Final	Q30	7	09 类、复制、继承与虚函数
2021 Spring Final	Q31	7、8	06 数组、地址步长与指针自增
2021 Spring Final	Q32（12 空）	8	06 数组、地址步长与指针自增
2022 Spring Final	Q1	2	05 对象、地址、指针与间接访问
2022 Spring Final	Q2	2	02 函数、参数、引用、const 与重载
2022 Spring Final	Q3	2	03 字符、C 字符串与 std::string
2022 Spring Final	Q4	2	04 流对象、引用返回与连续输出
2022 Spring Final	Q5	2	06 数组、地址步长与指针自增
2022 Spring Final	Q6	2	05 对象、地址、指针与间接访问
2022 Spring Final	Q7	2	05 对象、地址、指针与间接访问
2022 Spring Final	Q9	2	10 模板、迭代器与基础容器接口
2022 Spring Final	Q11	3	01 从类型出发读表达式
2022 Spring Final	Q12	3	08 生命周期、动态内存与 static
2022 Spring Final	Q14	3	07 进制与可寻址空间
2022 Spring Final	Q15	3、4	05 对象、地址、指针与间接访问

原卷	题号	原 PDF 页	专题
2022 Spring Final	Q17	4	11 边界补充：位运算提取颜色分量
2022 Spring Final	Q20	5	05 对象、地址、指针与间接访问
2022 Spring Final	Q21	5	09 类、复制、继承与虚函数
2022 Spring Final	Q22	5	02 函数、参数、引用、const 与重载
2022 Spring Final	Q23	5、6	10 模板、迭代器与基础容器接口
2022 Spring Final	Q24	6	09 类、复制、继承与虚函数
2022 Spring Final	Q25	6	08 生命周期、动态内存与 static
2022 Spring Final	Q26	6	05 对象、地址、指针与间接访问
2022 Spring Final	Q29	7	09 类、复制、继承与虚函数
2022 Spring Final	Q30	7	09 类、复制、继承与虚函数
2022 Spring Final	Q31	7、8	06 数组、地址步长与指针自增
2022 Spring Final	Q32（12 空）	8	06 数组、地址步长与指针自增

未纳入正文的原题仍保留在完整原卷中。两卷各有 8 道未选：

原卷与题号	排除理由
两卷 Q8	哈希碰撞及性能，系统内容在同教师 L13 Maps，超出主体
两卷 Q10	全面编程范式/函数式支持的概括判断，当前 11 讲没有足够直接课件定位
两卷 Q13	归并排序复杂度，系统内容为 L19-L20 算法分析/排序
两卷 Q16	minimax 博弈树，系统内容在 L21 Backtracking
两卷 Q18	BST 退化与遍历，系统内容在 L15 Trees
两卷 Q19	priority queue 的堆表示及下沉/上浮，L11 仅介绍接口和 vector 实现，不能据此把完整 heap 算法视作已覆盖
2021 Q28、2022 Q27	递归成立条件，独立章节为 L17-L18
2021 Q29、2022 Q28	环形缓冲区队列的实现判满；L11 的队列例为链表实现，完整环形数组实现不在该文件中

Q17 采用不同处理，是因为旧 M10 已包含位运算，且只需少量规则即可和本次进制主线相连；正文明确将它列为边界补充，并上传 L14 对应摘录，不伪装成 U6 已系统讲过位运算。

课件回查：本讲没有用原题覆盖的其他内容

下表用于补全复习面：找到的试卷没有问到某内容，不代表该内容不考。它列的是已读课件包含的内容及回查位置，不冒充额外的 exam 原题。

Lecture	本专题之外还应回查的内容
U01	程序结构与命名空间 p16-24；作用域 p29-30；短路逻辑 p49-51；分支/循环 p52-61；风格与 assert p62-64
U02	GCD 与 Euclid p11-16；默认参数 p19-21；调用过程与交换 p24-35；运算符重载 p36-41
U03	编译与链接 p6-7；头文件/实现 p12-13；Direction 与前后缀 ++ p14-19；常量导出与接口原则 p20-23；随机数库 p24-34
U04	cctype p9；C 字符串库与容量 p10-15；string 方法与下标 p27-34；大小写转换、回文、缩写 p35-38
U05	格式化与操纵符 p6-18；文件流和失败状态 p19-32；文件/字符串区别 p33-34；stringstream p35-39
U06	对象指针、->、this p30-33；C 字符串函数 strlen/strcpy p39-44；指针交换和引用对照 p27-29
U07	动态数组/二维数组 p7-13；析构与 RAII p14-19；const 成员初始化 p22-23；类 static p25-26；智能指针 p27-30
S08	struct/class 与 Point p5-20；初始化列表 p29-30；运算符 p31-39；Rational p43-53；TokenScanner p54-56；多继承与虚继承 p62-64
S09	函数模板 p5-10；类模板及 Stack p11-24；实现迭代器 p29-32
S10	ADT 接口与表示 p5-9；vector p10-16；括号配对的栈应用 p17-23；反向迭代 p28-29（暂定边界补充）
S11	单/双链表结点与插入删除 p4-22；回文应用 p23-25、30-31；队列模拟 p26-33；priority queue 接口及应用 p34-38（暂定边界补充）

课件中的简化与样例错误不自动成为 C++ 规则。例如 U6 对左值的“能否赋值”简化要补上可修改性；S08 p28 深复制样例不能直接当成安全实现照抄；S09 p28 的集合迭代器概括不涵盖所有标准适配器；U5/旧卷中流一律只能引用返回的说法需要区分复制与移动。相应题目已说明边界。

必要的语言与库核对来源

以下一手资料用于核对语言/库边界，不用于代替本校的真实考试范围：

- C++ 工作草案：basic_ostream 声明与复制限制
- C++ 工作草案：文件输入流构造和移动
- C++ 工作草案：range-based for 的展开
- C++ 工作草案：移位运算（这是当前草案，不倒推旧标准所有规则）
- Stanford 作者维护的旧版 Stack 头文件
- Stanford 作者维护的旧版 Queue 头文件

学习时每次先独立写出结果，再检查推导。对代码题，真正可迁移的是判断类型、对象身份和每条语句状态的方法；只记住两年的答案数字无法处理下一份卷子的局部改动。